

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: **WEB TECHNOLOGY**

Subject Code: **CS T63**

UNIT-II

Server Side Programming: servlet – strengths – Architecture – Life cycle – Generic and HTTP servlet – Passing Parameters – Server Side Include – Cookies – Filters.

JSP – Engines – Syntax – Components – Scriptlets – JSP Objects – Actions – Tag Extensions – Session Tracking – Database connectivity. Sql statements – J2EE – Introduction – Beans – EJB.

2 MARKS

1. Compare Client and Server Side Programming?

(APR 2016)

- In the client-side programming paradigm, programs/scripts are downloaded, interpreted, and executed by the browser.
- Applets are client-side programming technology, where special Java programs are embedded directly into web pages using <applet> tag. When a browser loads such a web page, the applet byte code is also downloaded and executed on the client side.
- **Example:** JavaScript, VBScript, HTML (Structure), CSS(Designing), AJAX, JQuery
- Server-side programming paradigm is that programs are executed in the web server. The web server sends web pages (containing simple code generated by those programs).
- **Example:** Java, JSP, PHP, Python, Ruby

2.What is a Servlet?

(NOV 2017)

- Servlet is a technology i.e. used to create web application.
- Servlet is java technology for server-side programming.
- Servlet is a Java module that runs inside a Java-enabled web server and services requests obtained from the web server.
- Servlets are not tied to a specific client-server protocol, but are most commonly used with HTTP.
- Servlet is a web component that is deployed on the server to create dynamic web page.

3. Write the four steps for execution of a Servlets?

1. The client sends a request to the web server.
2. The web server interprets it and forwards it to the corresponding servlet.
3. The servlet processes the request, generates the output and send it back to the web server.
4. The web server sends the response back to the client. The browser then displays it on the screen.

4. What are the advantages over applets?

- Applets, upon downloaded from the server, runs in the client's browser.
- It will function properly, provided a proper Java Runtime Environment (JRE) is installed in the client's browser.
- If the client uses old fashioned browsers and the applets use advanced features, the browsers may fail to execute them.
- Servlets run on the server machine and usually generate simple HTML codes.
- Even an older version of the browser can display these HTML pages correctly.
- Servlets run on the server machine and usually generate simple HTML codes.
- Even an older version of the browser can display these HTML pages correctly.

5. List the several alternatives to Servlets?

- Common Gateway Interface (CGI)
- Proprietary API
- Active Server Pages (ASP)
- Server-Side JavaScript

6. What is Common Gateway Interface (CGI)?

- Common Gateway Interface (CGI) is one of the most common server side solutions.
- CGI application is an independent program that receives requests from the web server and send it back to the web server.
- CGI specification is independent of any programming language.
- Most of the web servers have built-in support for CGI.

7. What are the problems in CGI technology?

- If number of client's increases, it takes more time for sending response.
- For each request, it starts a process and Web server is limited to start processes.
- It uses platform dependent language e.g. C, C++, perl.

8. List the advantages of Java Servlet technology?

- Efficient
- Persistent
- Portable
- Robust
- Extensible
- Secure
- Cost-effective

9. List the various Standalone web server?

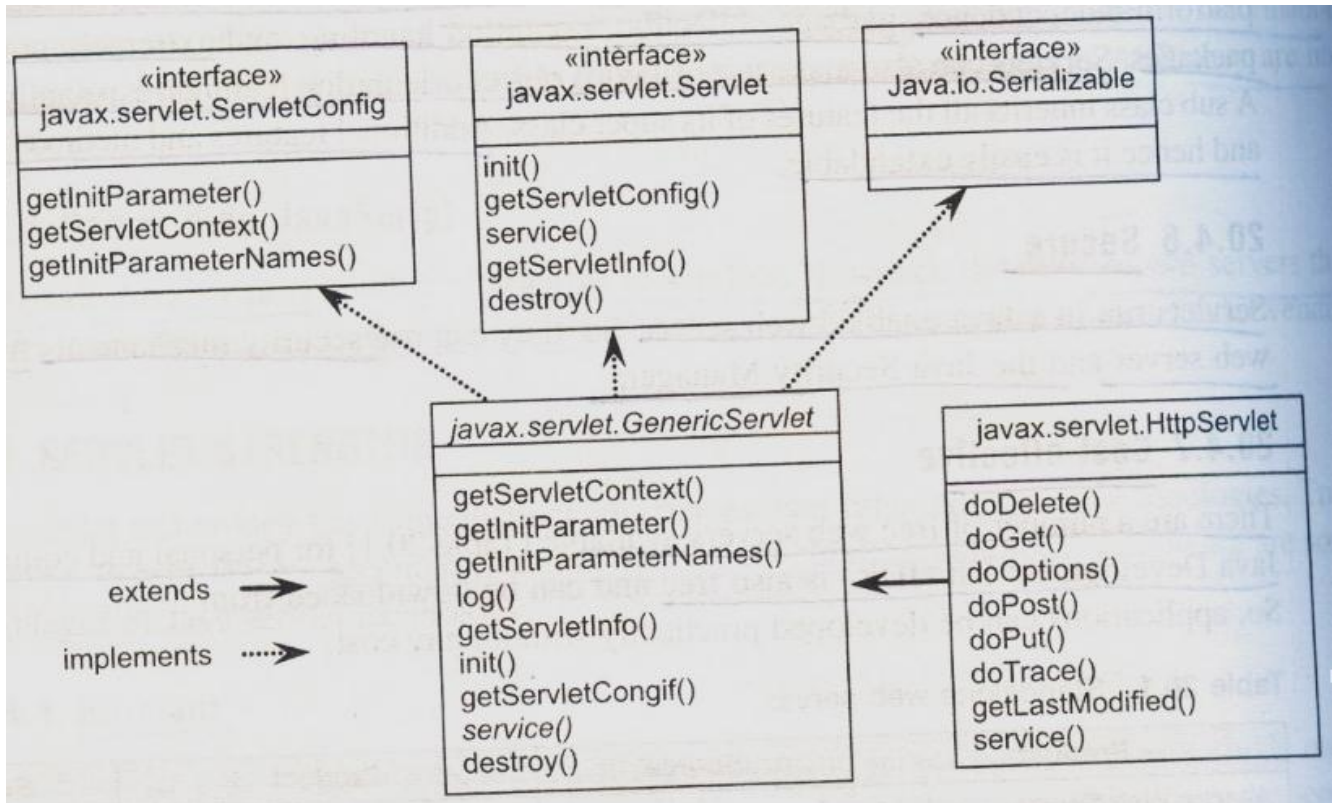
Product	Source
Apache Web Server	Apache
WebSphere Application Server	IBM
Oracle Application Server	Oracle
Web Server	ZEUS
J2EE Server	Pramati
JRUN	Adobe
Jigsaw Server	W3C

10. What is Servlet Architecture?

Servlet architecture consists of two packages:

- ✓ javax.servlet
- ✓ javax.servlet.http
- The first package consists top-level interfaces and classes that are used and extended by all other servlets.
- The second package is provided for servlets that can handle HTTP requests.

11. Draw the Servlet Architecture?



12. List the life cycle of a Servlet?

1. The `init()` method initializes the servlet
2. The `service()` method serves the client request and
3. The `destroy()` method shuts the servlet down.

13. List the steps involved in Servlet life cycle?

1. Servlet class is loaded.
2. Servlet instance is created.
3. `init()` method is invoked.
4. `service()` method is invoked.
5. `destroy()` method is invoked.

14. What are the types of Servlet?

There are two types of servlets,

- i) GenericServlet
- ii) HttpServlet

- **GenericServlet** defines the generic or protocol independent servlet.
- **HttpServlet** is subclass of GenericServlet and provides some http specific functionality like doGet and doPost methods.

15. What is meant by GenericServlet?

- GenericServlet class defines a generic protocol-independent servlet, it can be extended to provide implementation of any protocol, such as HTTP, FTP and SMTP.
- GenericServlet class can handle any type of request so it is protocol-independent.
- The GenericServlet class is created to make writing servlets easier.

16. What is meant by HttpServlet?

- The HttpServlet class extends GenericServlet and provide a framework for handling HTTP requests.
- Servlets that want to handle only HTTP requests are usually created by extending this class.
- It provides an implementation for the service() method.
- It provides the definition of HTTP specific methods such as doGet(), doPost(), doTrace() etc.

17. What is the difference between ServletContext and ServletConfig?

- ✓ **ServletContext:** Defines a set of methods that a servlet uses to communicate with its servlet container. The ServletContext object is contained within the ServletConfig object, which the web server provides the servlet when the servlet is initialized.
- ✓ **ServletConfig:** The object created after a servlet is instantiated and its default constructor is read. It is created to pass.

18. How to passing Parameters to a Servlet?

There are two ways in which you can pass parameters to a Servlet:

1. Using URL and
2. Using HTML forms

19. Define Server-Side Include (SSI)?

- ❑ Server Side Includes (SSI) is a simple interpreted server-side scripting language.
- ❑ SSI is to include the contents of one or more files into a web page on a web server.
- Server-Side Include (SSI) allows us to embed (include) servlets within HTML pages.
- The HTML files containing servlets typically have the extension .shtml.
- The web server understands this extension and forwards it to an internal servlet.

- The internal servlet parses the HTML document and executes the referenced servlets and sends the result back to the web server after merging it with the original HTML document.

20. Write the syntax for SSI servlet?

Tomcat SSI directives have the following syntax:

```
<!-- #directive attribute="value" attribute="value" ..... -->
```

21. List the various SSI directives?

- ✓ echo
- ✓ printenv
- ✓ config
- ✓ fsize
- ✓ flastmod
- ✓ include
- ✓ set

22. What is cookie? How to create it?

(NOV 2016)

- Cookies are one of the solutions to session tracking.
- A cookie is key-value pair created by the server and is installed in the client's browser when the client makes a request for the first time. Browser also maintain a list of cookies installed in them and send them to the server as a part of a subsequent HTTP requests. The server can then easily identify that this request is a part of a sequence of related requests. This way, cookie provide an elegant solution to session tracking.
- To create a cookie

Cookie (String key, String value)

23. What is the use of cookies?

- Cookies are short pieces of data sent by web servers to the client browser.
- The cookies are saved to clients' hard disk in the form of small text file.
- Cookies help the web servers to identify web users, by this way server tracks the user.
- Cookies pay very important role in the session tracking.

24. List the various Cookie class?

Cookie has

- a name,
- a single value, and
- optional attributes such as a
 - ✓ comment,
 - ✓ path and domain qualifiers,
 - ✓ a maximum age, and
 - ✓ a version number

25. List the limitations of Cookies?

1. Cookies can carry small pieces of information and are not a standard way of communication.
2. Some web browsers limit the number of cookies (typically 20 per web server) that can be installed. To avoid this problem, more than one block of information may be sent per cookie.
3. The value of a cookie should never exceed 4KB.
4. Cookie cannot identify a particular user.
5. Cookies cannot differentiate between multiple browsers running in a single computer.
6. Intruder can snoop, steal cookies and attack sessions. This is called session hijacking.

26. Define Filters?

- A filter is an object that is invoked at the preprocessing and post processing of a request. It is mainly used to perform filtering tasks such as conversion, logging, compression, encryption and decryption, input validation etc.
- Filters are objects that are installed between the client and server to inspect requests and responses. They can transform the request or modify the response. Filters process requests before they reach a servlet or process responses after leaving a servlet.

27. What is the purpose of Filter?

1. Intercept and inspect requests before dispatching them to the servlets.
2. Modify request's header and data and discard or filter requests.
3. Intercept and inspect responses before dispatching them to the servlets.
4. Modify request's header and data and discard or filter responses.

28. Define Sandbox?

A Sandbox is a container of servlets where restrictions are imposed. The administrator of the web server decides which servlets are which permissions, so that they cannot compromise's with the system's security.

29. Compare Servlet with CGI?

Servlets have several advantages over CGI:

- A Servlet does not run in a separate process. This removes the overhead of creating a new process for each request.
- A Servlet stays in memory between requests. A CGI program (and probably also an extensive runtime system or interpreter) needs to be loaded and started for each CGI request.
- There is only a single instance which answers all requests concurrently. This saves memory and allows a Servlet to easily manage persistent data.

30. Define JSP?

Java Server Pages (JSP) is a Java technology that allows software developers to dynamically generate HTML, XML or other types of documents in response to a Web client request. The technology allows Java code and certain pre-defined actions to be embedded into static content.

- ✓ Java Server Pages (JSP) is an extension of servlet technology.
- ✓ JSP simplify the delivery of dynamic web content.
- ✓ They allow web programmers to create dynamic content by reusing predefined components and by interacting with components using server-side scripting.

Java Server Pages (JSP) is a server-side technology that enables web programmers to generate web pages dynamically in response to client requests. There are many sever side technologies for building dynamic web applications. JSP is nothing but high-level abstraction of Java servlet technology. It allows us to directly embed pure Java code in an HTML page.

31. What is meant by Web Container?

JSP runs under the supervision of an environment called web container. The web container compiles the page on the server and generates a servlet, which is loaded in the Java Runtime Environment (JRE). The servlet serves the client requests in the usual way. This makes the development of the web applications convenient, simple and fast.

32. What is the advantages of JSP?

1. JSP technology provides excellent server-side programming for web applications that need database access.
2. JSP provides cross web server and cross platform support, but also integrates the WYSIWYG (What You See IS What You Get) features of static HTML pages with the extreme power of Java technology.
3. JSP allow us to separate the dynamic content of our web pages from the static HTML content.
4. JSP uses HTML, XML and Java code, the application are secure, fast and independent of server platforms.

33. What is the use of JSP Engines?

JSP engine is nothing but a specialized servlet, which runs under the supervision of the servlet engine. The JSP engine is typically connected with a web server or can be integrated inside a web server or an application server.

34. How JSP works?

1. Translates the JSP source code into the servlet source code.
2. It compiles the servlet source code to generate a class file.
3. Loads the class files and create an instance.
4. Initializes the servlet instance by calling the `jspInit()` method.
5. Invokes the `jspService()` method, passing the request and response objects.

35. What are the Life cycles of JSP?

- A web server is responsible for initializing, invoking and destroying.
- A web server communicates with a servlet through a simple interfaces as
 - ✓ JspInit()
 - ✓ JspService()
 - ✓ JspDestroy()

36. Difference between JSP and Servlet?

JSP	Servlet
JSP is a webpage scripting language that can generate dynamic content.	Servlets are Java programs that are already compiled which also creates dynamic web content.
JSP run slower compared to Servlet as it takes compilation time to convert into Java Servlets.	Servlets run faster compared to JSP.
It's easier to code in JSP than in Java Servlets.	Its little much code to write here.
JSP are generally preferred when there is not much processing of data required.	Servlets are best for use when there is more processing and manipulation involved.

37. Write the syntax of JSP?

The syntax of JSP is almost similar to that of XML. The general rules are

- a) Tags must have their matching end tags.
- b) Attributes must appear in the start tags.
- c) Attribute values in the tag must be quoted.

38. List out the JSP Components?

A JSP page consists of the following components:

1. Directives
2. Scripting Elements
 - ✓ Expressions
 - ✓ Declarations
 - ✓ Scriptlets
3. Standard Actions

39. List the various JSP tags?

JSP Tag	Meaning
<%@ %>	JSP directives such as page and include
<%= %>	JSP expressions
<% %>	JSP scriptlets that can contain arbitrary Java statements
<%! %>	Used for variables, methods and inner class declarations

40. List out various attributes for the page directives in JSP?

The page directive defines information that will be globally available for that Java Server Page,

1. language
2. extends
3. import
4. session
5. buffer
6. autoFlush
7. info
8. errorPage
9. contentType

41. What are the types of directives?

A Directives element in a JSP page provides global information about a particular JSP page. The syntax of jsp directive is

<%@ directive-name attribute="value" %>

The three standard directives available in all JSP environments:

1. page
2. include
3. taglib

42. Write the syntax of page directives?

The page directive defines attributes that notify the web container about the general settings of a JSP page.

It has the following syntax:

<%@ page attribute="value" attribute="value" %>

Example:

<%@ page language="java" %>

<%@ page language="java" import="java.sql.*" %>

43. Write the syntax of include directives?

- Include directives are used to specify the name of the files to be inserted during the compilation of the JSP page.
- This directive is to include the HTML, JSP or Servlet file into a JSP file.
- This is a static inclusion to a file.

The Syntax is

<%@ include file="filename" %>

Example:

```
<%@ include file="/header.html" %>
<%@ include file="/doc/legal/disclaimer.html" %>
<%@ include file="sortmethod" %>
```

44. What are the types of scripting elements?

The 3 different types of Scripting Elements are

1. Expressions
2. Declarations
3. Scriptlets

45. Write the syntax of Expressions?

The Syntax of JSP Expression is

<% = expression %>

- ✓ JSP Expression start with <% = and ends with %>.
- ✓ Between these, any number of expressions can be embedded.
- ✓ The result of expression is converted to the string to get displayed.

Example:

```
<%= "Hello World!" %>
```

46. Write the syntax of declarations?

The syntax of JSP Declaratives is

```
<%!
    // declare all the variables here
%>
```

- ✓ JSP declaration begins with <%! and ends with %>.
- ✓ We can embed any amount of java code in the JSP declarations.
- ✓ Variables and functions in the declarations are class level and can be used anywhere in the JSP page.

47. What is the use of Scriptlets?

- We can embed any amount of java code in the JSP Scriptlets.
- JSP Engine places these codes in JspService () methods.

Variables available to the JSP Scriptlets are

- ✓ Request – HttpServletRequest
- ✓ Response – HttpServletResponse
- ✓ Session – HTTP session object associated with the request
- ✓ Out - is an object of output stream and is used to send any output to the client.

48. List the out the JSP Implicit objects?

- ✓ Out
- ✓ Request
- ✓ Response
- ✓ Config
- ✓ Session
- ✓ Application

49. What is the use of Standard Actions?

JSP actions are used to perform tasks, such as insertion of files, forwarding a user to another page and instantiating objects.

The syntax for the JSP standard action is

```
<jsp:action>  
.....  
</jsp:action>
```

50. List the various Standard Actions in JSP?

The commonly used JSP action tags

- ✓ Include
- ✓ Forward
- ✓ Plugin
- ✓ UseBean

51. List the JSP specification in tag extension?

- ✓ Simple Tags
- ✓ Tag with attributes
- ✓ Tags with body
- ✓ Tags Defining Scripting Variables
- ✓ Cooperating Tags

52. Write the basic steps in design and use of custom tags?

- ✓ Tag Definition
- ✓ Provide Tag Handler
- ✓ Deploy the tag
- ✓ Use the tag in the JSP file

53. What is meant by session tracking?

A session is basically a conversation between a browser and a server. All the above technologies can save information for the current session for a particular user visiting a site. The session is important, as HTTP is a stateless protocol. This means that the connection between web server and a web browser is not automatically maintained, and that the state of a web session is not saved.

54. Mention any two uses of Session tracking?**(NOV 2016)**

There are four techniques used in Session tracking:

1. Cookies
2. Hidden Form Field
3. URL Rewriting
4. Session API

55. Define JDBC?

Java Database Connectivity (JDBC) allows us to access databases through Java programs. It provides Java classes and interfaces to SQL and PL/SQL statements, process results and performs other operations common to database.

56. What is meant by JDBC drivers?

A Java application can access almost all types of databases such as relational, object, and object relational. The access to a specific database is accomplished using a set of Java interfaces, each of which is implemented by different vendors differently. A Java class that provides interfaces to a specific database is called JDBC driver.

57. List the types of JDBC drivers?

JDBC drivers are classified into four categories

1. JDBC-ODBC bridge
2. Native-API, Partly Java
3. Middleware, Pure Java
4. Pure Java Driver

58. Write the basic steps in database connectivity?

1. Loading a Driver
2. Making a connection
3. Executing an SQL statement

59. What is meant by J2EE?

The Java 2 Enterprise Edition (J2EE) provides a component-based technology for the design, development, assembly and deployment of low-cost and fast-track enterprise applications. The purpose of J2EE is to simplify the design and implementation of distributed enterprise applications.

60. Mention the functionalities in J2EE? J2EE

offers the following fundamentals are

- ✓ Multi-tiered distributed application model
- ✓ Reusable components
- ✓ Flexible transaction control
- ✓ Web services support through integrated data interchange on XML-based open standards and protocols
- ✓ Unified security model

61. What is meant by JavaBeans?

- JavaBean technology is a Java-based technology to design and develop reusable and platform-independent software components.
- These components can then be integrated virtually in all Java-compatible applications.
- JavaBean components are known as beans.

62. List the advantages of JavaBeans?

- ✓ Write Once
- ✓ Run Anywhere
- ✓ Use Everywhere

63. What is meant by EJB?

- Enterprise JavaBeans (EJB) is a Java-based component architecture for design and development of world-class distributed modular enterprise applications.
- EJB are platform independent, but it can run in any application server that implements EJB specifications.

64. Mention the components of EJB?

The EJB component architecture integrates several enterprise-level requirements such as

- distributions,
- transactions,
- security,
- messaging,
- persistence and Enterprise Resource Planning (ERP) systems

65. List the three kinds of beans in EJB architecture?

There are three kinds of beans in the EJB architecture:

1. Session Beans
2. Entity Beans
3. Message Driven Beans

66. What are the ways by which a server and a browser do communicate? (APR 2017)

The ways by which a server and a browser do communicate are

- HTTP
- DNS
- TCP/IP
- ISP
- GET/POST method
- Cookies
- Sessions
- Query Strings

67. List out any four major features of JSP. (NOV 2017)

- Platform independence
- Enhanced performance
- Separation of logic from display
- Ease of administration
- Ease of use

11 MARKS

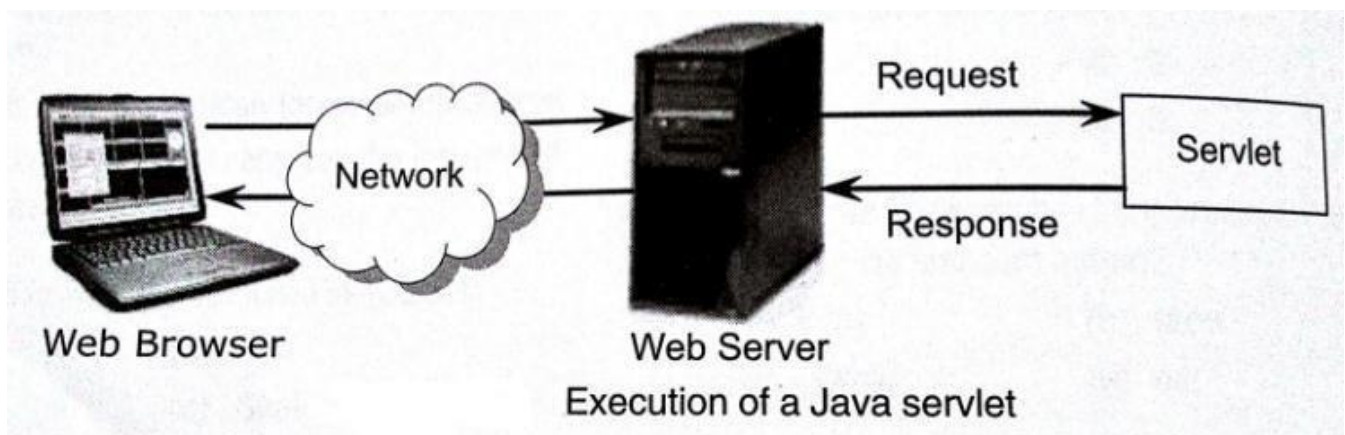
1. Explain briefly about Servlet?

(5 MARKS)

- Servlet is a technology i.e. used to create web application.
- Servlet is java technology for server-side programming.
- Servlet is a Java module that runs inside a Java-enabled web server and services requests obtained from the web server.
- Servlet is a class that extend the capabilities of the servers and respond to the incoming request.
- It can respond to any type of requests.
- Servlets are not tied to a specific client-server protocol, but are most commonly used with HTTP.
- Servlet is a web component that is deployed on the server to create dynamic web page.

Execution of a servlet consists of four steps:

1. The client sends a request to the web server.
2. The web server interprets it and forwards it to the corresponding servlet.
3. The servlet processes the request, generates the output and send it back to the web server.
4. The web server sends the response back to the client. The browser then displays it on the screen.



- Servlet runs entirely within the Java Virtual Machine (JVM).
- Since it runs on the server and generates output, which even older version of browsers can interpret, there is no browser incompatibility.

Advantages over Applets:

1. Applets, upon downloaded from the server, runs in the client's browser. It will function properly, provided a proper Java Runtime Environment (JRE) is installed in the client's browser. If the client uses old fashioned browsers and the applets use advanced features, the browsers may fail to execute them.

2. Servlets run on the server machine and usually generate simple HTML codes. Even an older version of the browser can display these HTML pages correctly.
3. Applets cannot access local resources such as file and database. Servlets, if configured properly, have full access to system resources. Therefore, servlets are more powerful than applets.

Servlet Alternatives

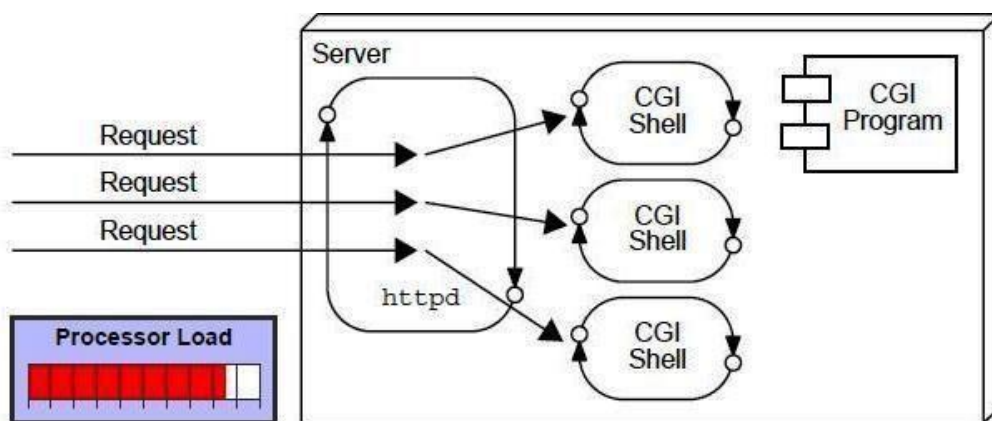
There are several alternatives to servlets. Some of the other server-side technologies are

1. Common Gateway Interface (CGI)
2. Proprietary API
3. Active Server Pages (ASP)
4. Server-Side JavaScript

1. CGI (Common Gateway Interface)

CGI scripting technology has a number of shortcomings, including platform dependence and lack of scalability. CGI application is an independent program that receives requests from the web server and send it back to the web server.

CGI technology enables the web server to call an external program and pass HTTP request information to the external program to process the request. For each request, it starts a new process.



Disadvantages of CGI

There are many problems in CGI technology:

1. If number of client's increases, it takes more time for sending response.
2. For each request, it starts a process and Web server is limited to start processes.
3. It uses platform dependent language e.g. C, C++, Perl.

2. Proprietary API

Many proprietary web servers have been built-in support for server-side programming. Examples include

- ✓ Netscape's NSAPI,
- ✓ Microsoft's ISAPI
- ✓ O'Reilly's WSAPI

Most of these are developed in C/C++ languages and hence can contain memory leaks and core dumps that can crash the web server.

3. Active Server Pages (ASP)

- Microsoft's Active Server Pages (ASP) is another technology that supports server-side Programming.
- Unfortunately, the only web server that supports this technology is Microsoft's Internet Information Server (IIS).
- Some third-party products support ASP, they are not free.

4. Server-Side JavaScript

- Server-Side JavaScript is another alternative to Servlets.
- However the only known servers that support it are Netscape's Enterprise and Fast Track Servers.

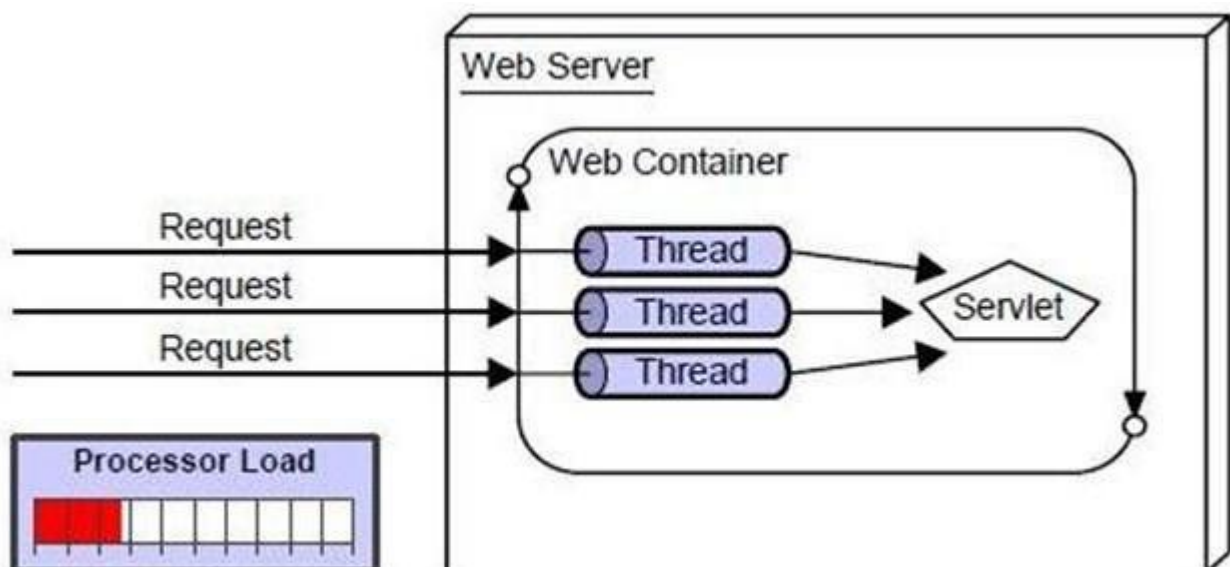
2. Write a short note on Servlet Strengths?

(6 MARKS)

The advantages of Java Servlet technology are

- ✓ Efficient
- ✓ Persistent
- ✓ Portable
- ✓ Robust
- ✓ Extensible
- ✓ Secure
- ✓ Cost-effective

The web container creates threads for handling the multiple requests to the servlet.



1. Efficient

When a servlet gets loaded in the server, it remains in the server's memory as a single object instance. Each new request is then served by a light-weight java thread spawned from the servlet.

2. Persistent

Servlets can maintain the session by using the session tracking and cookies. This mechanism that helps them track information from request to request. Critical information can also be saved to a persistent storage and can be retrieved from it when the servlet is loaded next time.

3. Portable

Servlets are written in Java and so they are portable across OS and server implementations. Servlets are Write Once Run Anywhere (WORA) programs.

4. Robust

Java has a very powerful exception handling mechanism and provides garbage collectors to handle memory leaks. Servlets are managed by JVM. It has class library with network and file support, database access, distributed object components, DOM support, security, utility packages etc.

5. Extensible

Java technology popular its well-known features such as a platform independence, garbage collection, exception handling and utility library packages. Servlets can also use of Java's object orientation features especially inheritance. Additional features and methods can be added and hence it is easily extensible

6. Secure

Servlets run in a Java-enabled web server. So, they can use security mechanisms from the both web server and the Java Security Manager.

7. Cost-effective

There are a number of free web servers available for personal and commercial use. Java Development Kit (JDK) is also free and can be downloaded from <http://java.sun.com>. So, applications can be developed practically without any cost.

Standalone Web Server

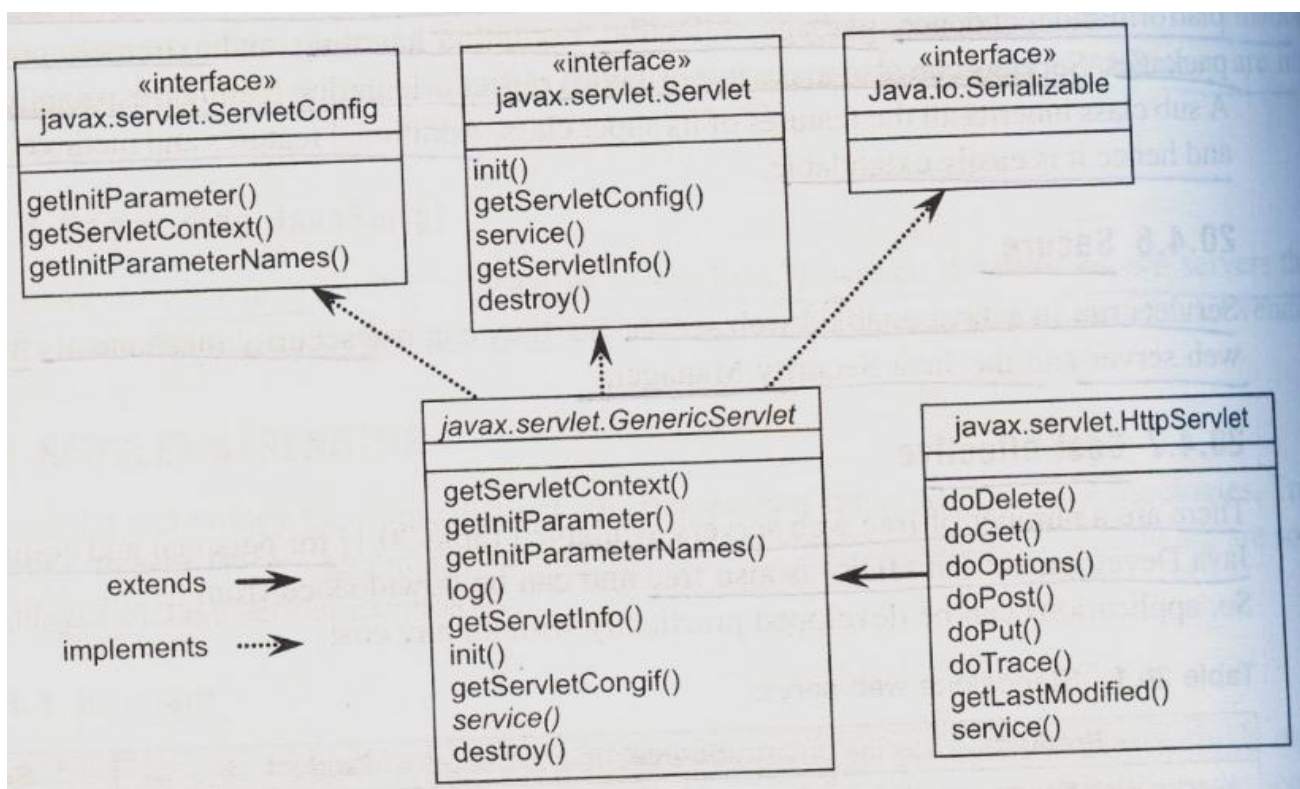
Product	Source	Product	Source
Apache Web Server	Apache	Web Server	ZEUS
WebSphere Application Server	IBM	iPlanet	Netscape
Weblogic Application Server	BEA	J Application Server	GenStone
Resin	Caucho	Netscape Enterprise Server	Netscape
JRUN	Adobe	LiteWebServer	Gefion
Orion Application Server	Orion	Java Web Server	Sun
Oracle Application Server	Oracle	CtO-JStar	JavaOne
Dynamo Application Server	ATG	iServer	ServerTec
J2EE Server	Pramati	Domino Go WebServer	Lotus
AppServer	Borland	Java Servlet Server 2.0	Paperclips
Jetty	Eclipse Foundation	KonaSoft Enterprise Server	KonaSoft
Jigsaw Server	W3C	Enhydra	ObjectWeb

3. Explain the Architecture of Servlet in detail?

(5 MARKS)

Servlet architecture consists of two packages:

- ✓ **javax.servlet**
- ✓ **javax.servlet.http**
- The first package consists top-level interfaces and classes that are used and extended by all other servlets.
- The second package is provided for servlets that can handle HTTP requests.



Servlet Architecture

The top-level interface of the servlet architecture is `javax.servlet.Servlet`. It provides the basic functionalities of all the servlets that are created by implementing this interface directly or indirectly.

For example,

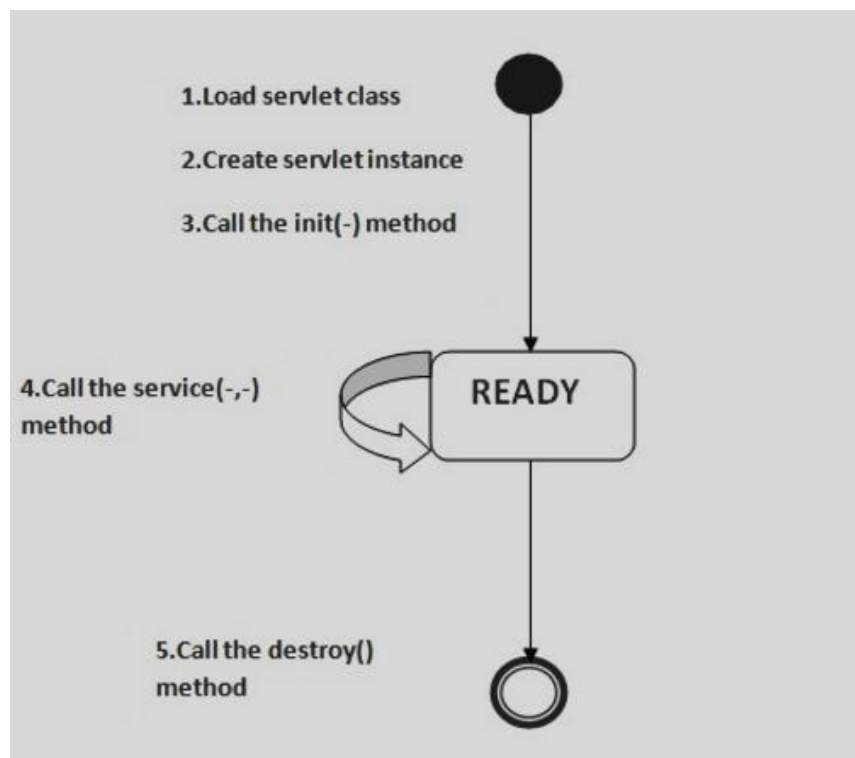
- ✓ The `init()` method initializes the servlet
- ✓ The `service()` method serves the client request and
- ✓ The `destroy()` method shuts the servlet down.

4. Explain the life cycle of servlet with a neat diagram? (11 MARKS) (NOV 2016) (APR 2017)

The web container in which the servlet has been deployed supervises and controls the life cycle of a servlet. Typically, this container is nothing but a web server.

When the web container receives a request from a client and determines that the requests should be handled by a servlet, it performs the following steps.

- i. Servlet class is loaded.
- ii. Servlet instance is created.
- iii. `init()` method is invoked.
- iv. `service()` method is invoked.
- v. `destroy()` method is invoked.



Life cycle of a Servlet

1. If an instance of the target servlet does not exist, the container does the following:
 - Finds and loads the servlet class
 - Creates an instance of the servlet class
 - Calls the `init()` method on this servlet instance to initialize it

2. Otherwise (if the target servlet exists), it invokes the `service()` method on it, passing a `ServletRequest` type object and a `ServletResponse` type object.
3. If the container decides that the servlet is no longer needed, it removes and finalizes the servlet by calling the servlet's `destroy()` method.

1. Servlet class is loaded

- The class loader is responsible to load the servlet class.
- The servlet class is loaded when the first request for the servlet is received by the web container.

2. Servlet instance is created

- The web container creates the instance of a servlet after loading the servlet class.
- The servlet instance is created only once in the servlet life cycle.

3. `init()` method is invoked

- The web container calls the `init()` method only once after creating the servlet instance.
- The `init()` method is used to initialize the servlet.
- It is the life cycle method of the `javax.servlet.Servlet` interface.

The syntax of the `init()` method is given below:

`public void init (ServletConfig config) throws ServletException`

The web container passes a `ServletConfig` object, which contains the startup configuration for this servlet such as initialization parameters.

4. `service()` method is invoked

- The `service()` is the most method in servlet,
`public void service (ServletRequest request, ServletResponse response) throws ServletException, IOException`

This method gets called each time the web container receives a request intended for this servlet. The web container calls the `service()` method on a servlet with two objects:

1. a `ServletRequest` type object request and
 2. a `ServletResponse` type object response
- ✓ The request object encapsulates the communication from the client to the server.
 - ✓ The response object encapsulates the communication from the servlet back to the client.
 - ✓ The `ServletRequest` interface provides methods to retrieve information sent by the clients.
 - ✓ The `ServletResponse` interface provides methods to send data to the clients.

5. `destroy()` method is invoked

- The web container calls the `destroy()` method before removing the servlet instance from the service.
- It gives the servlet an opportunity to clean up any resource for example memory, thread etc.

The syntax of the destroy() method of the Servlet interface is given below:

public void destroy()

Other Methods

The javax.servlet.Servlet interface also provides the following methods to inspect the properties of a servlet at runtime.

1. ServletConfig getServletConfig();

- Returns a ServletConfig object, which contains the initialization parameters and startup configuration used for this servlet.

2. String getServletInfo();

- Returns a String object containing information about the servlet, such as its version, author, copyright, etc.

5. Explain Generic Servlet and HTTP Servlet in detail?

(11 MARKS)

Generic Servlet:

- The class javax.servlet.GenericServlet implements the javax.servlet.Servlet interface and the javax.servlet.ServletConfig interface.
- A servlet class is usually created by extending either the GenericServlet class or its descendant javax.servlet.http.HttpServlet class.
- If a servlet does not need to be a subclass of another class, it must implement the servlet interface directly. This would be necessary when, for example, RMI or CORBA objects act as servlets. The servlet class must implement all the methods of the servlet interface.

GenericServlet class defines a generic protocol-independent servlet, it can be extended to provide implementation of any protocol, such as HTTP, FTP and SMTP.

GenericServlet class can handle any type of request so it is protocol-independent. The GenericServlet class is created to make writing servlets easier. It provides the implementations of the life cycle methods init() and destroy(), as well as methods in the ServletConfig interface. The servlet writer has to implement only the service() method. This method is called every time a client requests for the servlet.

The prototype of this method is

public void service (javax.servlet.ServletRequest request, javax.servlet.ServletResponse response) throws javax.servlet.ServletException, java.io.IOException;

This method has two arguments,

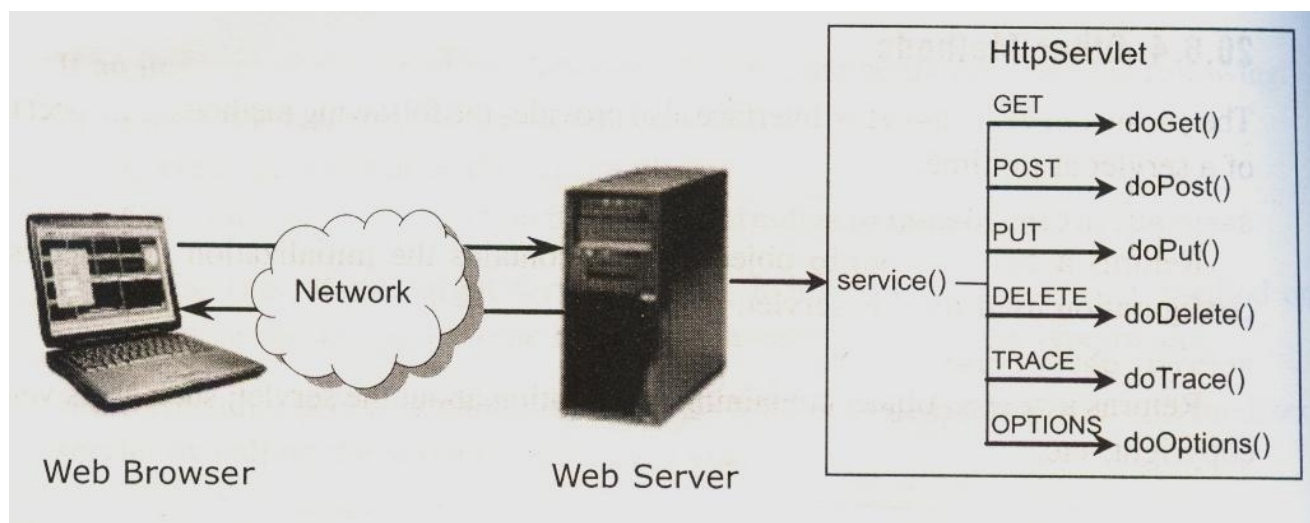
- Request encapsulates the client request and is used to extract information from the client's request.
- Response contains information to be sent back to the client.

Methods of Generic Servlet class

1. **public void init(ServletConfig config)** is used to initialize the servlet.
2. **public abstract void service(ServletRequest request, ServletResponse response)** provides service for the incoming request. It is invoked at each time when user requests for a servlet.
3. **public void destroy()** is invoked only once throughout the life cycle and indicates that servlet is being destroyed.
4. **public ServletConfig getServletConfig()** returns the object of ServletConfig.
5. **public String getServletInfo()** returns information about servlet such as writer, copyright, version etc.
6. **public ServletContext getServletContext()** returns the object of ServletContext.
7. **public String getInitParameter(String name)** returns the parameter value for the given parameter name.
8. **public Enumeration getInitParameterNames()** returns all the parameters defined in the web.xml file.
9. **public String getServletName()** returns the name of the servlet object.
10. **public void log(String msg)** writes the given message in the servlet log file.

HTTP Servlet:

- The HttpServlet class extends GenericServlet and provide a framework for handling HTTP requests. Servlets that want to handle only HTTP requests are usually created by extending this class. It provides an implementation for the service() method.
- The service() method in the HttpServlet class reads the method type stored in the request and invoke a specific method based on this value. Specifically, if the method type is GET, it calls doGet(); if the method type is POST, it calls doPost().



HttpServlet in action

Each of these methods takes two arguments: an `HttpServletRequest` type object and an `HttpServletResponse` type object.

The prototype of the `doGet()` method

`void doGet(HttpServletRequest request, HttpServletResponse response)`

The `HttpServletRequest` interface extends the `ServletRequest` interface and provides all the functionalities of the `ServletRequest` interface. Additionally, it provides methods to retrieve HTTP headers, cookies sent, and other HTTP-specific information.

The `HttpServletResponse` interface extends the `ServletResponse` interface and provides all the functionalities of the `ServletResponse` interface. Additionally, it provides methods to send HTTP-specific information back to the client.

Servlets created by extending the `HttpServlet` class can handle multiple simultaneous requests. For each request, a thread is created, which runs its `service()` method. If you want to create a single-threaded servlet, your servlet must also implement the `SingleThreadModel` interface as follows:

`public class SimpleServlet extends HttpServlet implements SingleThreadModel { }`

The interface `SingleThreadModel` does not define any method. It is used to declare that the servlet should use a single thread.

Methods

The methods of `HttpServlet` class are as follows:

1. **`protected void doGet (HttpServletRequest request, HttpServletResponse response)`**: It handles the GET request. It is invoked by the web container.
2. **`protected void doPost (HttpServletRequest request, HttpServletResponse response)`**: Similar to `doGet()` method, it also gets invoked by the web container and it handles the POST request.
3. **`public void service (ServletRequest request , ServletResponse response)`**: There are two types of service method, one is public and other one is protected. The purpose of this public method is to dispatch the request to the protected service method by converting the request and response object into HTTP type.
4. **`protected void service (HttpServletRequest request , HttpServletResponse response)`**: This method receives the request from the public `service()` method. It dispatches the request to the `doXXX()` method depending on the incoming http request type.
5. **`protected void doHead (HttpServletRequest request , HttpServletResponse response)`**: It handles the HEAD request.
6. **`protected void doOptions (HttpServletRequest request , HttpServletResponse response)`**: Performs the HTTP OPTIONS operation; the default implementation of this method automatically determines what HTTP Options are supported.

7. **protected void doPut (HttpServletRequest request , HttpServletResponse response):** Performs the HTTP PUT operation; the default implementation reports an HTTP BAD_REQUEST error.
8. **protected void doTrace (HttpServletRequest request , HttpServletResponse response):** Performs the HTTP TRACE operation; the default implementation of this method causes a response with a message containing all of the headers sent in the trace request.
9. **protected void doDelete (HttpServletRequest request , HttpServletResponse response):** Performs the HTTP DELETE operation; the default implementation reports an HTTP BAD_REQUEST error.
10. **protected long getLastModified (HttpServletRequest request):** Gets the time the requested entity was last modified; the default implementation returns a negative number, indicating that the modification time is unknown and hence should not be used for conditional GET operations or for other cache control operations as this implementation will always return the contents.

EXAMPLE:

A simple servlet designed to handle the HTTP GET method. Create the following servlet Helloworld.java using any text editor.

Helloworld.java:

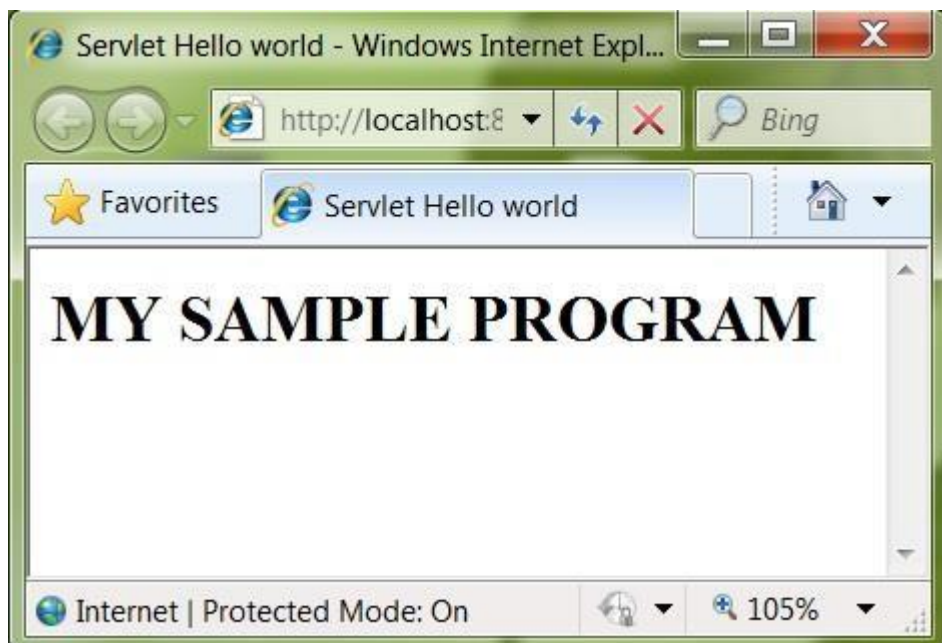
```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Helloworld extends HttpServlet
{
    public void doGet (HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println(" <html> ");
        out.println(" <head> ");
        out.println(" <title> Servlet Hello world </title> ");
        out.println(" </head> ");
        out.println(" <body> ");
        out.println(" <h1> MY SAMPLE PROGRAM </h1> ");
        out.println(" </body> ");
        out.println(" </html> ");
        out.close();
    }
}
```

web.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<servlet>
  <servlet-name> Helloworld </servlet-name>
  <servlet-class> Helloworld </servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name> Helloworld </servlet-name>
  <url-pattern> /Servlet/Helloworld </url-pattern>
</servlet-mapping>
```

OUTPUT:



6. Explain in detail how to passing a parameters to Servlets with an example? (11 MARKS)

There are two ways in which you can pass parameters to a Servlet:

1. Using URL
2. Using HTML forms

1. Passing Parameters directly to a Servlets

- Parameters can be passed to servlets directly using URL. In this case parameters and their values are attached directly to the URL when you use the URL to call the servlet.
- The URL parameters are separated by '?' character.
- The character ? says "here are the parameters".
- Each parameter takes the form name = value.
- Multiple parameters are separated by the '&' character.

Consider the following URL,

http://192.168.1.2:8080/servlet/check?login=abc&password=xyz

- In this example, two parameters are passed login and password whose values are abc and xyz, respectively.
- Certain special characters such as space and quote (") are not allowed directly in the URL.
- These characters are encoded in the %hh format where hh is the hex code of the character.
- For example, %20, %22 and %7E represent the space, quote and tilde (~) characters, respectively.

2. Passing Parameters directly to a Servlets using HTML Forms

One of the disadvantages of passing parameters using URLs is that users can view the parameter names with their values passed. So, it is not safe to send sensitive information using this method.

- Moreover, the number of parameters that we can pass is also limited.
- HTML forms can be used to pass parameters in a convenient way.

The following HTML code can be used to pass the same information

```
<form method="post" action="http://192.168.1.2:8080/servlet/check">  
    Login: <input type="text" name="login"> <br>  
    Password: <input type="password" name="pass">  
    <br> <input type="submit" value="Login">  
</form>
```

- The POST method is used here. In this case, the values of all the form fields are embedded in the body of the HTTP request message and hence cannot be viewed directly.
- However, servlets can retrieve them using the usual way.

Retrieving Parameters

The three methods of the ServletRequest interface may used to retrieve parameters. Their signatures are as follows

1. public abstract java.lang.String **getParameter**(java.lang.String);
 2. public abstract java.lang.String[] **getParameterValues** (java.lang.String);
 3. public abstract java.lang.Enumeration **getParameterNames**();
-
- The getParameter() method returns the value of the specified parameter. The parameter has exactly one value.
 - If there are multiple parameters with the same name, use the getParameterValues() method, which returns an array of string values of the specified parameter.
 - The getParameterNames() method returns all the parameters as an enumeration. The values of the individual parameters can be obtained by iterating through the enumeration and using the getParameter() method for each of these parameters.

EXAMPLE:

The following example shows how to retrieve values of two parameters, login and password, and return back to the client.

index.html:

```
<html>

    <head>
        <title> Login Page </title>
    </head>
    <body>
        <h1> LOGIN FORM </h1>
        <form method = "post" action = "Login">
            Login: <input type="text" name="login"> <br>
            Password: <input type="password" name="pass"> <br>
            <input type="submit" value="Login"> <input
            type="reset" value="Clear">
        </form>
    </body>
</html>
```

Login.java:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Login extends HttpServlet
{
    public void doPost(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head>");
        out.println("<title> Login Display Page </title>");
        out.println("</head>");
        out.println("<body> <center>");
        String login = request.getParameter("uname");
        String pass = request.getParameter("pword");
        out.println("<h1> Login: " + login + "<br> Password: " + pass + "</h1>");
        out.println(" </center> </body>");
        out.println("</html>");
        out.close();
    }
}
```

Compile the servlet, make the following entry in the web.xml file, restart the web browser.

web.xml:

```
<servlet>
    <servlet-name> Login </servlet-name>
    <servlet-class> Login </servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>Login</servlet-name>
    <url-pattern>/Login</url-pattern>
</servlet-mapping>
```

OUTPUT:

Passing Parameters



A screenshot of a Windows Internet Explorer browser window titled "Login Page". The address bar shows "http://localhost:8". The page content displays "LOGIN FORM" in large, bold, black letters. Below this, there are two input fields: "Login:" with the text "ADMIN" and "Password:" with masked characters "•••••". At the bottom of the form are two buttons: "Login" and "Clear". The browser's status bar at the bottom indicates "Internet | Protected Mode: On" and a zoom level of "130%".

Retrieving Parameters



A screenshot of a Windows Internet Explorer browser window titled "Login Display Page". The address bar shows "http://localhost:8". The page content displays the result of a login attempt in large, bold, black letters: "Login: ADMIN" and "Password: cse123". The browser's status bar at the bottom indicates "Internet | Protected Mode: On" and a zoom level of "130%".

Servlets return the parameter with their values back to the client. Typically they consult the database for further processing. The servlet will possibly verify the login name and password information against the information stored in the database and depending upon the result, either allow or disallow the user to login.

7. Explain in detail about Server-Side Include (SSI)?

(11 MARKS)

- Server-Side Include (SSI) allows us to embed (include) servlets within HTML pages.
- The HTML files containing servlets typically have the extension .shtml.
- The web server understands this extension and forwards it to an internal servlet.
- The internal servlet parses the HTML document and executes the referenced servlets and sends the result back to the web server after merging it with the original HTML document.
- This include dynamically generated content in an existing HTML file, without having to serve the entire page via a CGI program or other dynamic technologies.

To enable SSI support, remove the XML comments from around the SSI servlet and servlet-mapping configuration in \$TOMCAT_HOME/conf/web.xml.

The syntax for including servlets in HTML documents varies widely from web server to web server.

Tomcat SSI directives have the following syntax:

<!-- #directive attribute="value" attribute="value" -->

- Tomcat SSI directives are formatted like an HTML comment.
- If SSI is not correctly configured, the browser will ignore it.
- Otherwise, the directive will be replaced with its results.

The various SSI directives are

- 1) echo
- 2) printenv
- 3) config
- 4) fsize
- 5) flastmod
- 6) include
- 7) set

1.echo

- It inserts the value of a variable.
- For example, the following directive inserts the value of the local date.

<!-- #echo var = "DATE_LOCAL" -->

This generates the following result:

Friday, 18-JAN-2017 13:15:44 IST

There are a number of standard variables, including all the environment variables that are available to servlets.

2. printenv

- It returns the list of all defined variables.

```
<!-- #printenv -->
```

It results in the following

```
HTTP_USER_AGENT=Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 5.1; Trident/4.0)
HTTP_ACCEPT_LANGUAGE=en-us HTTP_ACCEPT=/*/* LAST_MODIFIED=11/20/09
DOCUMENT_URI=/wt/SSI.shtml HTTP_ACCEPT_ENCODING=gzip, deflate REMOTE_PORT=1482
SERVER_NAME=192.168.1.2 SERVER_SOFTWARE=Apache Tomcat/6.0.16 Java HotSpot (TM)
Client VM/11.0-b12 Windows XP SCRIPT_FILENAME=D:\ApacheTomcat6.0.16 on
Thinkpad\webapps\wt\SSI.shtml DATE_LOCAL=11/20/09 SERVER_ADDR=192.168.1.2
SERVER_PROTOCOL=HTTP/1.1 REQUEST_METHOD=GET DOCUMENT_NAME=SSI.shtml
SERVER_PORT=8080 SCRIPT_NAME=/SSI.shtml REMOTE_ADDR=192.168.1.2
DATE_GMT=11/20/09 REMOTE_HOST=192.168.1.2 HTTP_HOST=192.168.1.2:8080
HTTP_CONNECTION=Keep-Alive UNIQUE_ID=2BD24DA4115256FAA76B7E55168AFC28
QUERY_STRING= GATEWAY_INTERFACE=CGI/1.1 org.apache.catalina.ssi.SSIServlet=true
REQUEST_URI=/wt/SSI.shtml
```

3. config

- The config directive, with a timefmt attribute, to modify the formatting of the date and time as follows:

```
<!-- #config timefmt = " %A %B %d, %Y " -->
```

```
<!-- #echo var="DATE_LOCAL" -->
```

This generates the following result:

Friday March 18, 2017

4. fsize

- This directive returns the size of the file specified by the file attribute.

```
<!-- #fsize file="SSI.shtml" -->
```

- This directive returns the size of the file SSI.shtml in kilobytes.

5. flastmod

- The last modification time of a file is inserted using flastmod directive whose file attribute specifies the file name.
- The following code inserts the last modification time of the file named SSI.html.

```
<!-- #flastmod file="SSI.shtml" -->
```

It is better to use the echo directive to print the value of the LAST_MODIFIED environment variable, which specifies the last modification time of the file where this directive is included.

```
<!-- #echo var="LAST_MODIFIED" -->
```

6. include

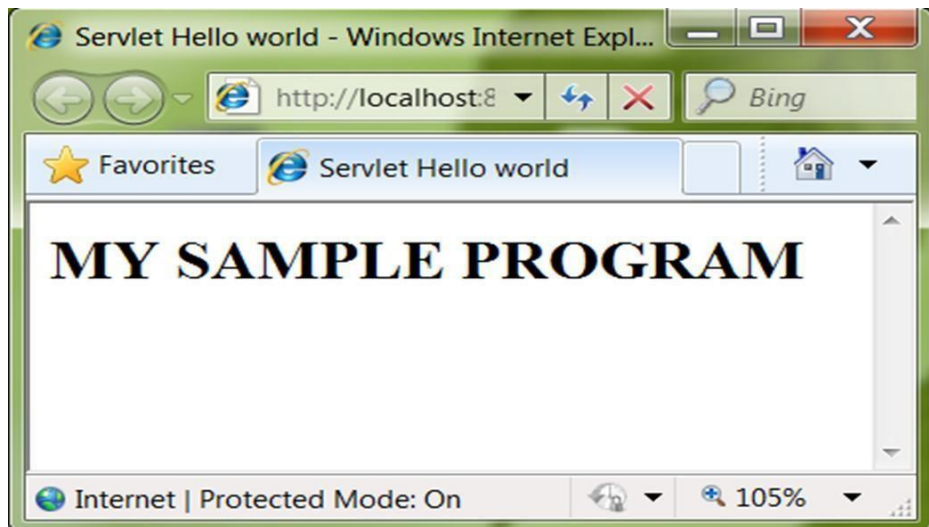
- The most common use of SSI is to include the result of a server-side program such as servlet or CGI. The following directives includes the result of our Helloworld servlet in the place of the directive.

```
<!-- #include virtual = "servlet/Helloworld" -->
```

The attribute virtual specifies the server-side script to be used. So, Create a file SSI.shtml containing a single line as follows and put in the \$TOMCAT_HOME \ webapps \ wt directory.

```
<!-- #include virtual = "servlet/Helloworld" -->
```

It generates the output as



The include directive is useful if you want to insert the result of the same servlet in different pages. Use the following to insert a header in every page.

```
<!-- #include virtual = "/header.html" -->
```

7. set

- It is used for creating variables and setting their values for later use.
- The following directive creates a variable SSI with the value "Server Side Include".

```
<!-- #set var ="SSI" value="Server Side Include" -->
```

- Similarly, the following code creates a variable sum with the value "0".

```
<!-- #set var ="sum" value="0" -->
```

8. Explain in detail about Cookies with an example?

(11 MARKS) (NOV 2017)

HTTP is stateless. Every HTTP request is different from others. Sometimes, it is necessary to keep track of a sequence of related requests sent by a client to perform some designated tasks. This is called **session tracking**.

A **cookie** is a small piece of information that is persisted between the multiple client requests. A cookie has

- a name,
- a single value, and
- optional attributes such as a
 - ✓ comment,
 - ✓ path and domain qualifiers,
 - ✓ a maximum age, and
 - ✓ a version number.

A **Cookie** is a small piece of data that is created by the server and it is stored in the client browser and that can later be read back from that browser. It is the information that is persisted between the multiple client requests.

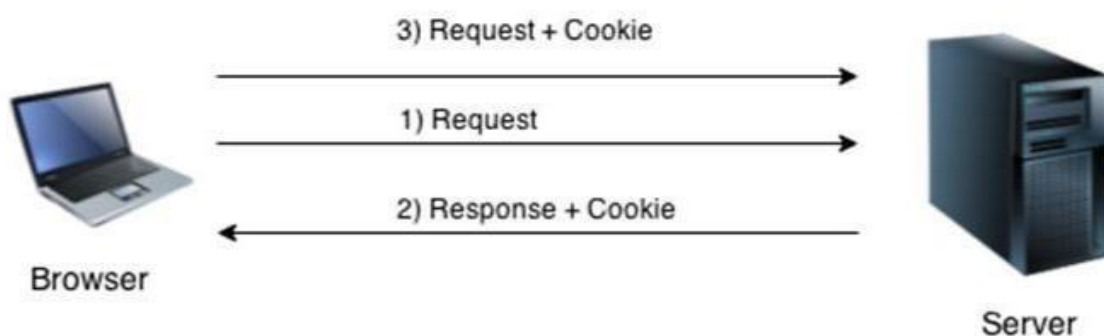
Cookies are one of the solutions to session tracking. A **cookie** is key-value pair created by the server and is installed in the client's browser when the client makes a request for the first time. Browser also maintain a list of cookies installed in them and send them to the server as a part of a subsequent HTTP requests. The server can then easily identify that this request is a part of a sequence of related requests. This way, cookie provide an elegant solution to **session tracking**.

The servlet API supports cookie. A cookie is represented using the **javax.servlet.http.Cookie class** and is created using the following constructor:

Cookie (String key, String value)

How Cookie works

- By default, each request is considered as a new request.
- In cookies technique, we add cookie with response from the servlet.
- So cookie is stored in the cache of the browser.
- After that if request is sent by the user, cookie is added with request by default.
- Thus, we recognize the user as the old user.



Types of Cookie

1. Non-persistent cookie

It is valid for single session only. It is removed each time when user closes the browser.

2. Persistent cookie

It is valid for multiple session. It is not removed each time when user closes the browser. It is removed only if user logout or sign-out.

Methods of Cookie class:

Method	Description
public void setMaxAge (int expiry)	Sets the maximum age of the cookie in seconds.
public String getName()	Returns the name of the cookie.
public String getValue()	Returns the value of the cookie.
public void setName (String name)	changes the name of the cookie.
public void setValue (String value)	changes the value of the cookie.

A cookie is added by the **addCookie()** method of the HttpServletResponse class. Similarly, the server can get all cookies sent by the web browser using the **getCookies()** method of the HttpServletRequest class.

EXAMPLE:

CookieDemo.java:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieDemo extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException,
    IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        Cookie[] cookies = request.getCookies();
        boolean found = false;
        if(cookies!=null) {
            for(int i=0 ; i < cookies.length; i++) {
```

```

        if(cookies[i].getName().equals("Session_Started")) {
            found = true;

            out.print("You started this session on : ");
            out.println(cookies[i].getValue());
        }
    }
}
if(!found)
{
    String dt = (new java.util.Date()).toString();
    response.addCookie(new Cookie("Session_Started", dt));
    out.println("Welcome to out site");
}
}
}

```

When a client makes the call for the first time, the web browser does not send any cookie. The servlet looks for the cookie with the name "Session_Started". Naturally, it cannot find the cookie. It creates, install, and sends a cookie with the name "Session_Started" using the following code.

response.addCookie(new Cookie("Session_Started", dt));

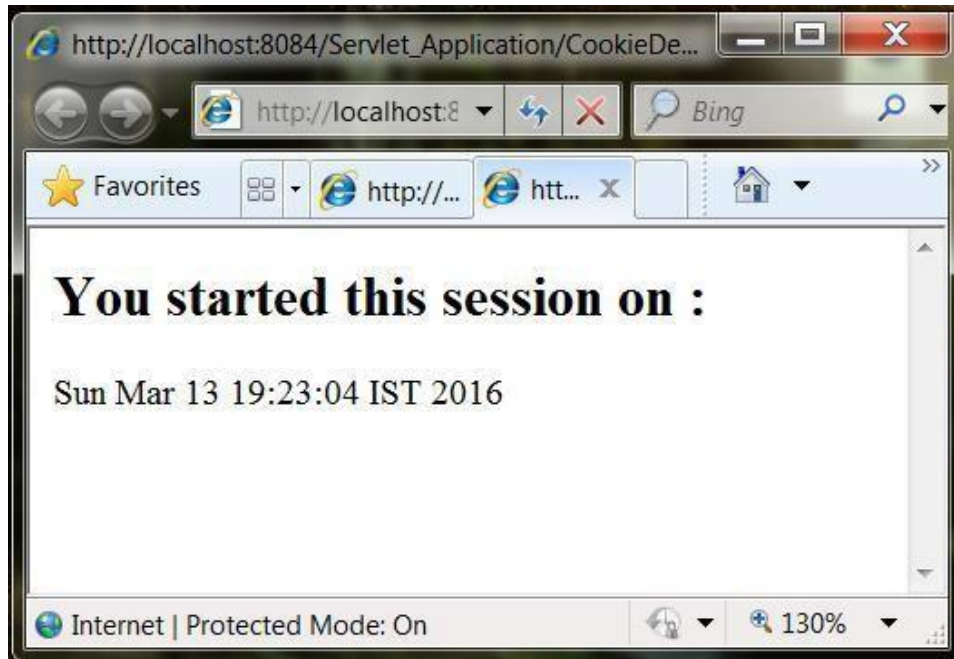
For subsequently calls, web browsers send this cookie with a request. Our servlet finds it, and retrieve the start time of this session and finally send this value.

OUTPUT:

(i) When you access this servlet for the first time



(ii) Any subsequent requests generates the output as



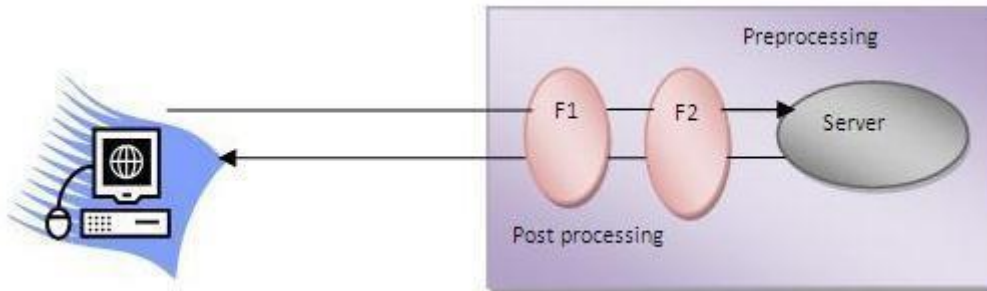
Limitations of Cookies

1. Cookies can carry small pieces of information and are not a standard way of communication.
2. Some web browsers limit the number of cookies (typically 20 per web server) that can be installed. To avoid this problem, more than one block of information may be sent per cookie.
3. The value of a cookie should never exceed 4KB. If the value of a cookie is larger than 4KB, it should be trimmed to fit.
4. Cookie cannot identify a particular user. A user can be identified by a combination of user account, web browser and computer. So, user who have multiple accounts and use multiple computers/ web browsers have multiple sets of cookies. Consequently, cookies cannot differentiate between multiple browsers running in a single computer.
5. Intruder can snoop, steal cookies and attack sessions. This is called session hijacking.

9. Explain briefly about Filter with an example?

(11 MARKS)

A filter is an object that is invoked at the preprocessing and post processing of a request. It is mainly used to perform filtering tasks such as conversion, logging, compression, encryption and decryption, input validation etc. The servlet filter is pluggable, i.e. its entry is defined in the web.xml file, if we remove the entry of filter from the web.xml file, filter will be removed automatically and we don't need to change the servlet. So maintenance cost will be less.



Usage of Filter

- Recording all incoming requests
- Logs the IP addresses of the computers from which the requests originate
- Conversion
- Data compression
- Encryption and decryption
- Input validation etc...

Filters are objects that are installed between the client and server to inspect requests and responses. They can transform the request or modify the response. Filters process requests before they reach a servlet or process responses after leaving a servlet.

A filter can do the following:

- Intercept and inspect requests before dispatching them to the servlets.
- Modify request's header and data and discard or filter requests.
- Intercept and inspect responses before dispatching them to the servlets.
- Modify request's header and data and discard or filter responses.

A filter can work on behalf of a single servlet or a group of servlets. Similarly, zero or more servlets can be installed for a servlet.

A filter class must implement the **javax.servlet.Filter** interface, which provides a framework for the filtering mechanisms. It defines the following method to be implemented by each filter class:

void init(FilterConfig config)

The web container call this method once to install it and to set its configuration object. The filter can then start servicing. The **FilterConfig** interface defines methods to retrieve the filter's name, its **init** parameters and the underlying servlet context.

void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)

This method is called before dispatching the request to the servlet. It receives the current request as well as the **FilterChain** containing filters that will handle this request after it. In this method, the filter should process the request and take necessary actions. The filter should then call the chain.doFilter() method to handover the control to the next filter in this chain. Whenever the response comes back, this filter can do additional tasks on the response.

If the filter decides, it may not forward the request to the next filter, in this case, the request will not reach the servlet at all. In this way, filters can drop malicious requests intended for the servlet.

void destroy()

This method is called to uninstall the filter.

The following filter forward only those requests that come from the host having the IP address "192.168.1.2". Requests from all other hosts are discarded.

EXAMPLE:

Firewall.java:

```
import java.io.*;
import javax.servlet.*;
public class Firewall implements Filter
{
    private FilterConfig config = null;
    public void init (FilterConfig config) throws ServletException
    {
        this.config=config;
    }
    public void doGet (ServletRequest request, ServletResponse response, FilterChain chain) throws
    ServletException, IOException
    {
        if (request.getRemoteAddr().equals("192.168.1.2"))
            chain.doFilter (request, response);
    }
    public void destroy()
    {
        config=null;
    }
}
```


Deploying Filter:

To use this filter, you must specify it in the web.xml file using the <filter> tag.

```
<filter>
    <filter-name> myFirewall </filter-name>
    <filter-class> Firewall </filter-class>
</filter>
```

This notifies the server that a filter named myFirewall is implemented in the Firewall class. Now, add the following lines in the web.xml file

```
<filter-mapping>
    <filter-name> myFirewall </filter-name>
    <url-pattern> /Helloworld </url-pattern>
</filter-mapping>
```

This code states that a filter has to be installed for the URL pattern /HelloWorld that corresponds to our HelloWorld servlet. Now, try accessing the HelloWorld servlet from the host 192.168.1.2. A response will be received as before. But, if the servlet is accessed from any machine other than 192.168.1.2, no response is received.

Filter API

The javax.servlet package contains the three interfaces of Filter API.

1. Filter
2. FilterChain
3. FilterConfig

1) Filter interface

For creating any filter, you must implement the Filter interface. Filter interface provides the life cycle methods for a filter.

Method	Description
public void init (FilterConfig config)	init() method is invoked only once. It is used to initialize the filter.
public void doFilter (HttpServletRequest request, HttpServletResponse response, FilterChain chain)	doFilter() method is invoked every time when user request to any resource, to which the filter is mapped. It is used to perform filtering tasks.
public void destroy()	This is invoked only once when filter is taken out of the service.

2) FilterChain interface

The object of FilterChain is responsible to invoke the next filter or resource in the chain. This object is passed in the doFilter method of Filter interface. The FilterChain interface contains only one method: **public void doFilter(HttpServletRequest request, HttpServletResponse response)**: it passes the control to the next filter or resource.

Advantage of Filter

- Filter is pluggable.
- One filter don't have dependency onto another resource.
- Less Maintenance

10. Discuss basic concepts in Java Server Pages (JSP)?

(6 MARKS)

- **JSP** abbreviated as **Java Server Pages**.
- JSP is Sun's solution for developing dynamic web sites.
- JSP provide excellent server side scripting support for creating database driven web applications.
- JSP is a server side technology that enables web programming to generate webpages dynamically in response to client requests.
- JSP uses HTML, XML and Java code, the application are secure, fast and independent of server platforms.
- JSP is nothing but high level abstraction of java servlet technology.
- It allows us to directly embed pure java code in an HTML page.
- JSP pages run under the supervision of an environment called web container.
- The web container compiles the page on the server and generates a servlet, which is loaded in the Java Runtime Environment (JRE).
- JSP enables the developers to directly insert java code into JSP file.
- JSP pages are efficient, they load into the web server's memory on receiving the request at the very first time and the subsequent calls are served within a very short period of time.
- This makes the development of the web applications convenient, simple and fast.
- Maintenance also becomes very easy.
- In today's environment, most website server's dynamic pages are based on user request.
- Database is very convenient way to store the data of users.
- JSP file have the extension .jsp

Java Server Pages (JSP) is a server-side technology that enables web programmers to generate web pages dynamically in response to client requests. There are many sever side technologies for building dynamic web applications.

JSP is nothing but high-level abstraction of Java servlet technology. It allows us to directly embed pure Java code in an HTML page. JSP runs under the supervision of an environment called web container. The web container compiles the page on the server and generates a servlet, which is loaded in the Java Runtime Environment (JRE). The servlet serves the client requests in the usual way. This makes the development of the web applications convenient, simple and fast. Maintenance also becomes very easy.

JSP technology provides excellent server-side programming for web applications that need database access. JSP provides cross web server and cross platform support, but also integrates the **WYSIWYG** (What You See IS What You Get) features of static HTML pages with the extreme power of Java technology.

JSP allow us to separate the dynamic content of our web pages from the static HTML content. JSP uses HTML, XML and Java code, the application are secure, fast and independent of server platforms. JSP file have the extension .jsp

JSP and ASP

- JSP is portable or running the entire platform. Asp runs only in MS windows platform.
- Similar operation

JSP and JavaScript

JSP

- ✓ It dynamically Web Pages can be changed on the server.
- ✓ It can access server side resources like database, catalogs, pricing information.
- ✓ It can access session tracking and cookies.

JavaScript

- ✓ It can generate dynamically on the client side.
- ✓ It does not access this type of resources.
- ✓ It does not have.

11. Explain JSP Engines and its working model?

(6 MARKS)

JSP Engines

- ✓ To Process JSP page, a JSP engine is needed.
- ✓ JSP Engine is nothing but a specialized servlet, which runs under the supervision of the servlet engine.
- ✓ The JSP Engine is typically connected with a web server or can be integrated inside a web server or an application server.
- ✓ Many servers are freely available and can be downloaded for evaluation or development of web applications.

Many servers are

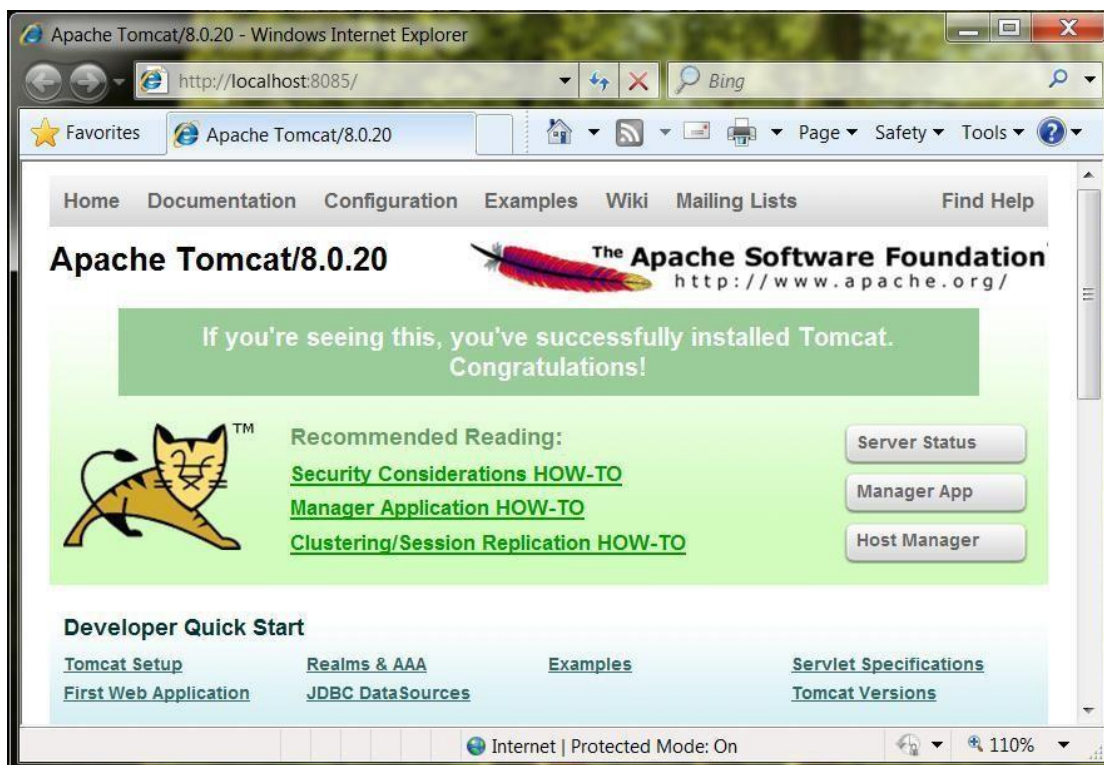
1. Tomcat
2. Java Web Server
3. WebLogic and
4. WebSphere

1. Tomcat

Tomcat implements servlet and JSP specifications. It is very easy to install and can be used as a small stand-alone server for developing and testing servlets and JSP pages. For large applications, it is integrated into the Apache Web Server.

- Tomcat is running on port no 8080 by default.
- You can test it by using the URL <http://127.0.0.1:8080/>

Apache Tomcat Home page



2. Java Web Server

The Java System Web Server by Sun is a leading web server that delivers secure infrastructure for medium and large business technologies and applications. Sun delivers 8x better performance than Apache Tomcat. It is available on all major OS and supports a technologies such as JSP and Java Servlet, PHP and CGI.

3. WebLogic

WebLogic by BEA systems of San Jose, California, is a J2EE application server and also an HTTP server for Microsoft Windows, Unix/Linux, and other platforms. WebLogic supports DB2, Oracle, Microsoft SQL Server and other JDBC databases.

The WebLogic server also includes the .NET framework for interoperability and allows integration of the following native components:

- ✓ CORBA connectivity
- ✓ COM+ Connectivity
- ✓ J2EE Connector Architecture
- ✓ IBM's WebSphere MQ Connectivity
- ✓ Native JMS messaging for enterprise

Data Mapping functionality and Business Process Management are also included in WebLogic Sever Process Edition.

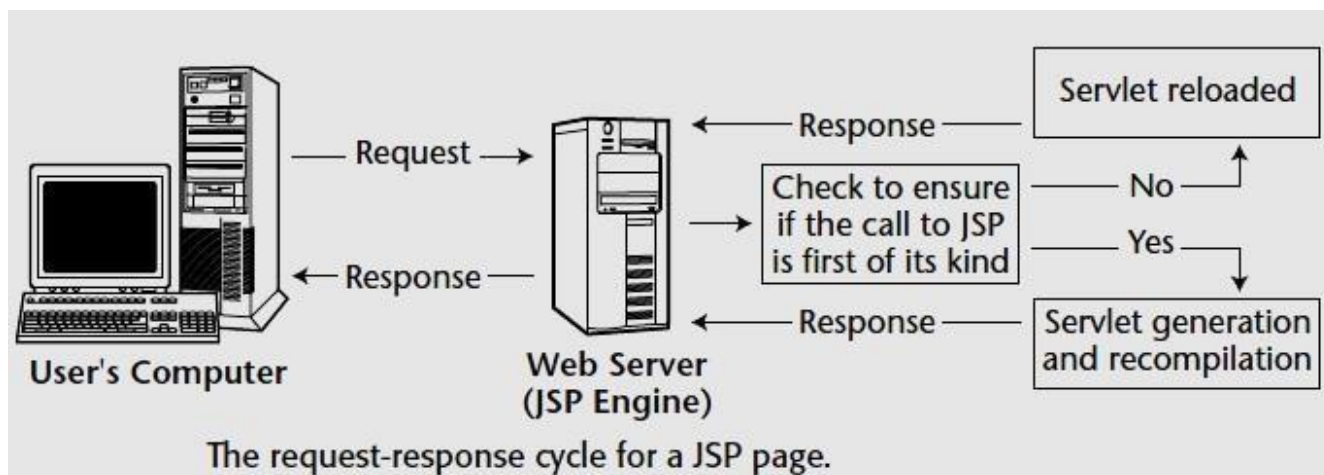
4. WebSphere

IBM attempt to develop a software to set up, operate and integrate electronic business applications that can work across multiple computing platforms, using Java-based web technologies. The result is WebSphere Application Server (WAS). It includes both the run-time components and the tools that can be used to develop robust and versatile applications that will run on WAS.

How JSP works?

When a client such as a web browser sends a request to a web server for a JSP file using a URL, the web server identifies the .jsp file extension in the URL and the requested resource is a Java Server Pages. The web server hands over the request to a special servlet. This servlet checks whether the servlet corresponding to this JSP page exists or not. If the servlet does not exist, but it is older than the JSP page, it performs the following:

1. Translates the JSP source code into the servlet source code.
2. Compiles the servlet source code to generate a class file.
3. Loads the class file and create an instance.
4. Initializes the servlet instance by calling the `jspInit()` method.
5. Invokes the `_jspService()` method, passing the request and response objects.



If the servlet exists and is not older than the corresponding JSP page requested, it does the following:

1. If an instance is already running, it simply forwards the request to this instance.
2. Otherwise, it loads the class file, create an instance, initializes it, and forwards the request to this instance.

Advantages

During development, one of the advantages of JSP pages over servlets is that the build process is automatically performed by the JSP engine. When a JSP file is requested for the first time, or if it is changed, the translation and compilation phase occurs. The subsequent requests for the JSP page directly go to the servlet byte code, which is already in the memory.

Life Cycle of a Servlet and JSP

- A web server is responsible for initializing, invoking and destroying.
- A web server communicates with a servlet through a simple interfaces as
 1. JspInit()
 2. JspService()
 3. JspDestroy()

12. Explain the components of JSP in detail?

(11 MARKS) (APR 2016).

JSP Syntax

The syntax of JSP is almost similar to that of XML. The general rules are

1. Tags must have their matching end tags.
2. Attributes must appear in the start tags.
3. Attribute values in the tag must be quoted.

White spaces within the body text of a JSP page are preserved during the translation phase. To use special characters such as '%', add a '\' character before it. To use the '\' character, add another '\' character before it.

JSP Components

A JSP page consists of the following components:

1. Directives
2. Scripting Elements
 - ✓ Expressions
 - ✓ Declarations
 - ✓ Scriptlets
3. Standard Actions
4. Custom Tags

JSP Tags

JSP tags	Meaning
<%@ %>	JSP directives such as page and include
<%= %>	JSP expressions
<% %>	JSP scriptlets that can contain arbitrary Java statements
<%! %>	Used for variables, methods and inner class declarations

1. Directives

- A Directives element in a JSP page provides global information amount a particular JSP page.
- The syntax of jsp directive is

<%@ directive-name attribute="value" %>

The three standard directives available in all JSP environments:

1. Page
2. include
3. Taglib

Page Directives:

- The page directive defines attributes that notify the web container about the general settings of a JSP page.
- It has the following syntax:

<%@ page attribute="value" attribute="value" %>

**<%@ page [language="java"]
[extends="package.class"]
[import="package.*",...]
[session="true|false"]
[buffer="none|8kb|sizekb"]
[autoflush="true|false"]**

```

[isThreadSafe="true|false"]
[info="text"]
[contentType="text/html"]
[isErrorPage="true|false"]

%>

```

Example:

```

<%@ page language="java" %>

<%@ page language="java" import="java.sql.*" %>

```

Page Directives Attributes

Attribute	Value
language	The language used in scriptlets, expressions, and declarations. In JSP 1.1, the only valid value for this attribute is java.
extends	The fully qualified name of the superclass of this JSP page. This must be a class that implements the <code>HttpJspPage</code> interface. The JSP specification warns against the use of this attribute without fully understanding its implications.
import	A comma-separated list of one or more <i>package.*</i> names and/or fully qualified class names. This list is used to create corresponding import statements in the generated Java servlet. The following packages are automatically included and need not be specified: <code>java.lang.*</code> <code>java.servlet.*</code> <code>java.servlet.jsp.*</code> <code>java.servlet.http.*</code>

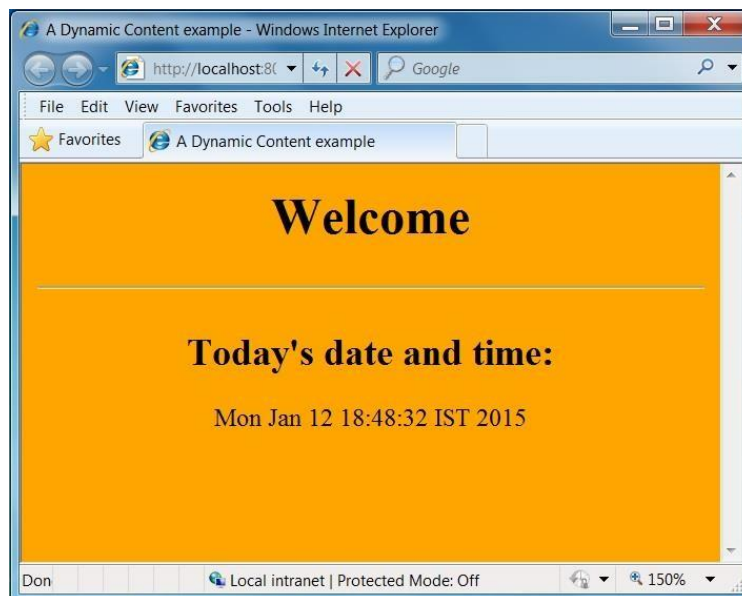
session	<i>true</i> or <i>false</i> , indicating whether the JSP page requires an HTTP session. If the value is <i>true</i> , then the generated servlet will contain code that causes an HTTP session to be created (or accessed, if it already exists). The default value is <i>true</i> .
buffer	Specifies the size of the output buffer. Valid entries are <i>nnnkb</i> or <i>none</i> , where <i>nnn</i> is the number of kilobytes allocated for the buffer. The default value is 8kb.
autoflush	<i>true</i> if the buffer should be automatically flushed when it is full, or <i>false</i> if a buffer overflow exception should be thrown. The default value is <i>true</i> .

Attribute	Value
isThreadSafe	<i>true</i> if the page can handle simultaneous requests from multiple threads, or <i>false</i> if it cannot. If <i>false</i> , the generated servlet declares that it implements the <code>SingleThreadModel</code> interface.
info	A string that will be returned by the page's <code>getServletInfo()</code> method.
isErrorPage	<i>true</i> if this page is intended to be used as another JSP's error page. In that case, this page can be specified as the value of the <code>errorPage</code> attribute in the other page's page directive. Specifying <i>true</i> for this attribute makes the <i>exception</i> implicit variable available to this page. The default value is <i>false</i> .
errorPage	Specifies the URL of another JSP page that will be invoked to handle any uncaught exceptions. The other JSP page must specify <code>isErrorPage="true"</code> in its page directive.
contentType	Specifies the MIME type and, optionally, the character encoding to be used in the generated servlet.

Page Directives Examples

```
<%@ page language="java" %>
<html>
    <head>
        <title> Simple JSP example </title>
    </head>
    <body>
        <h1> Welcome </h1> <hr>
        <h3>Today's date and time: </h3>
        <%-- This is the JSP content that displays the server time by using the method Date() --%>
        <% =new java.util.Date() %>
    </body>
</html>
```

Output



Include Directives

- Include directives are too used to specify the name of the files to be inserted during the compilation of the JSP page.
- This directive is to include the HTML, JSP or Servlet file into a JSP file.
- This is a static inclusion to a file.

Syntax

```
<%@ include file="filename" %>
```

Example

```
<%@ include file="/header.html" %>
<%@ include file="/doc/legal/disclaimer.html" %>
<%@ include file="sortmethod" %>
```

Include Directives Examples

first.jsp

```
<html>

    <head>

        <title> An Include example </title>

    </head>

    <body>

        <h2> This is the content of first.jsp </h2> <hr>

        <% ="Hello, welcome to the world of Java Server Pages!!!" %>

        <%@ include file="second.jsp" %>

    </body>

</html>
```

Second.jsp

```
<html>

    <head>

        <title> An Include example </title>

    </head>

    <body>

        <h2> This is the content of second. Jsp </h2> <hr>

        <%= "Hope you're having fun learning JSP" %>

    </body>

</html>
```

Output



Comments

- The JSP comment syntax is
`<%- - This is a hidden JSP comment - - %>`
- and the latter looks like this:
`<!-- This is included in the generated HTML -->`

2. Scripting Elements

The 3 different types of Scripting Elements are

1. Expressions
2. Declarations
3. Scriptlets

Expressions

Syntax of JSP Expression is

`<% = expression %>`

- JSP Expression start with `<% =` and ends with `%>`.
- Between these, any number of expressions can be embedded.
- The result of expression is converted to the string to get displayed.

Example

`<%=“Hello World!” %>`

Expression examples

```
<html>
    <head>
        <title> My First JSP Example </title>
    </head>

    <body>
        <h1> My First JSP Example </h1> <hr>
        <%-- This is the JSP content --%>
        <%=“Hello World” %>
    </body>
</html>
```

Declarations:

The syntax of JSP Declaratives is

```
<%!  
    // declare all the variables here  
%>
```

- JSP declaration begins with <%! and ends with %>.
- We can embed any amount of java code in the JSP declarations.
- Variables and functions in the declarations are class level and can be used anywhere in the JSP page.

Scriptlets:

The Syntax of JSP Scriptlets

```
<%  
    //java code  
%>
```

Example:

```
<%  
    String username=cse;  
    Username = request.getParameter("username");  
%>
```

Program 1:

```
<%@ page language="java"%>  
<html>  
    <head>  
        <title> Count program in JSP </title>  
    </head>  
    <body>  
        <%!  
            intcnt=0;  
            privateintgetcount()  
            {  
                // increment cnt and return the value  
                cnt++;  
                returncnt;  
            }  
        %>  
        <p> The Values of cnt are:  
        <%=getcount()%>  
        <%=getcount()%>
```

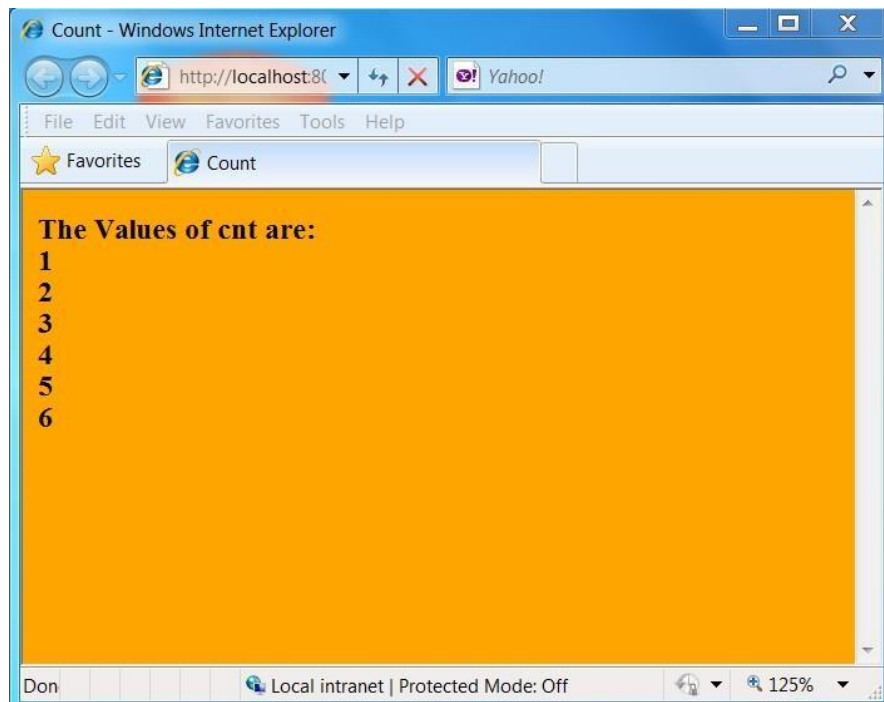
```

        <%=getcount()%>
        <%=getcount()%>

    </body>
</html>

```

Output



- We can embed any amount of java code in the JSP Scriptlets.
- JSP Engine places these codes in JspService() methods.
- Variables available to the JSP Scriptlets are
 1. **Request** – HttpServletRequest
 2. **Response** – HttpServletResponse
 3. **Session** – HTTP session object associated with the request
 4. **Out** - is an object of output stream and is used to send any output to the client.

3. Custom Tags

- The taglib directive makes custom actions available in the current page through the use of a tag library.

The syntax of the directive is

```
<%@ taglib uri="tagLibraryURI" prefix="tagPrefix" %>
```

Attribute	Value
tagLibraryURI	The URL of a Tag Library Descriptor.
tagPrefix	A unique prefix used to identify custom tags used later in the page.

13. Explain the JSP objects in detail?

(5 MARKS)

In a JSP page, objects may be created using directives, action, or scriptlets. JSP object is defined as the availability of that object for use from a particular place of the web application. There are four object scope

1. page
2. request
3. session and
4. application

Page object

Objects having page scope can be accessed only from within the same page where they created. JSP implicit objects out, exception, response, pageContext, config and page have page scope.

request object

A request can be served by more than one page. Objects having request scope can be accessed from any page that serves that request.

session object

Objects having session scope are accessed from pages that belong to the same session from where they were created.

application object

JSP objects have the application scope can be accessed from any page that belong to the same application.

Implicit Objects

Web container allows us to directly access objects defined in the `_jspService()` method of the JSP page's underlying servlet. These objects are called implicit objects as they are instantiated automatically. The implicit objects contain information about request, response, session, configuration etc.

Variable	Class	Description
Out	javax.servlet.jsp.JspWriter	Output stream of the JSP page's servlet
request	javax.servlet.HttpServletRequest	Current client request being handled by the JSP page
response	javax.servlet.HttpServletResponse	Response generated by the JSP page to be returned to the client
config	javax.servlet.ServletConfig	Initialization information of the JSP page's servlet
session	javax.servlet.HttpSession	Session object for the client

application	javax.servlet.ServletContext	Context of the JSP page's servlet
exception	javax.lang.Throwable	Represents error
page	Java.lang.Object	Refers to JSP page's servlet processing the current request
PageContext	javax.servlet.jsp.PageContext	JSP page request, response, session, application and servlet configuration

14. Write a short notes on Standard Actions in JSP?

(11 MARKS)

The JSP engine provides many built-in sophisticated functions to the programmers for easy development of the web applications. These functions include instantiating objects in a JSP page communicating with other JSP page and servlets.

JSP actions is nothing but XML tags that can be used in a JSP page to use this functions.

JSP actions are used to perform tasks, such as

- ✓ insertion of files,
- ✓ Forwarding a user to another page and
- ✓ instantiating objects.

The syntax for the JSP standard action is

```
<jsp:action>
.....
</jsp:action>
```

The various JSP action tags are

1. include
2. forward
3. param
4. plugin
5. useBean

1. include

It is used to include a file in a JSP page. The general syntax is

```
<jsp:include page= "relativeURL | <%= expression %>" flush = "true" />
```

Example: The following code includes the file header.jsp in the current page:

```
<jsp:include page="header.jsp" />
```

It is similar to the include directive but instead of inserting the text of the included file in the original file at compilation time, it actually includes the target at run-time. It acts like a subroutine where the control is passed temporarily to the target. The control is then returned back to the original JSP page. The result of the included file is inserted in the place of <jsp:include> action in the original JSP page.

Example:

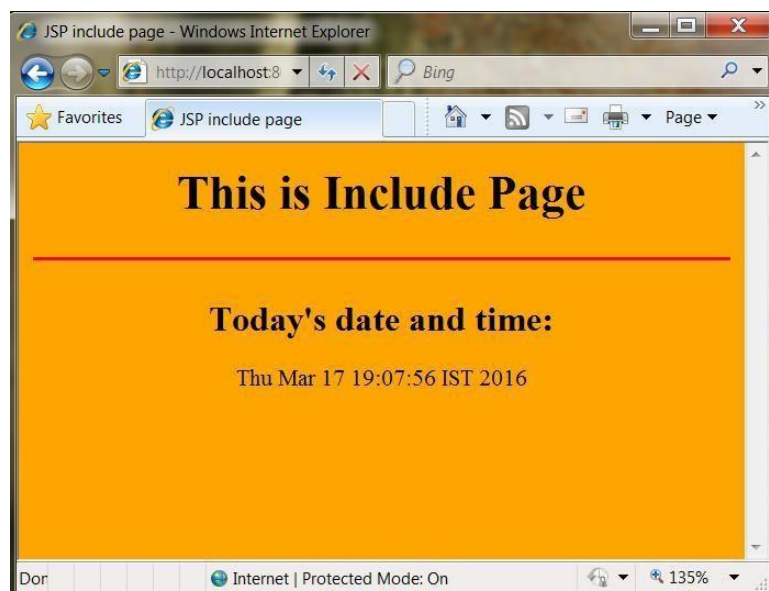
Include.jsp

```
<%@ page language="java" %>
<html>
    <head>
        <title> JSP include page </title>
    </head>
    <body>
        <%-- using the <jsp:include> action --%>
        <h1 align="center"> This is Include Page </h1>
        <hr color="red">
        <jsp:include page ="date.jsp" flush = "true" />
    </body>
</html>
```

Date.jsp:

```
<%@ page language="java" import="java.util.*" %>
<html>
    <head>
        <title> A Dynamic Content example </title>
    </head>
    <body bgcolor = "orange">
        <center>
            <h2> Today's date and time: </h2>
            <%= new Date() %>
        </center>
    </body>
</html>
```

Output:



2.forward

- This action tag hands over the current request to the specified page internally at the server side.
- This will redirect to the different page without notifying browser.

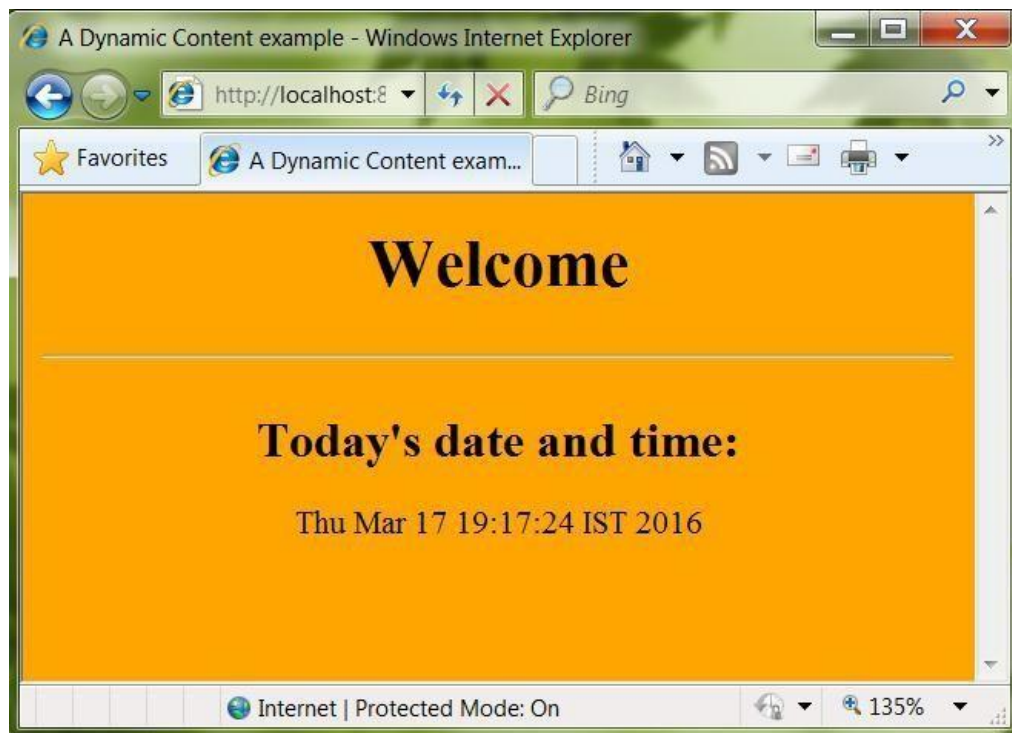
The general syntax of JSP forward action tag is

```
<jsp:forward page = "filename" | <%= expression %>" />
```

Example:

```
<%@ page language = "java" %>
<html>
    <head>
        <title> A forward action example </title>
    </head>
    <body>
        Before <br>
        <jsp:forward page = "date.jsp" />
        <br> After
    </body>
</html>
```

Output:



3.param

- The JSP <jsp:param> action allow us to append additional parameters to the current request.
- The general syntax is

<jsp:param name ="parameterName" value ="parameterValue" />

- The name and value attributes of the <jsp:param> tag specify the case sensitive name and value of the parameter respectively.

It is typically used with the <jsp:include> and <jsp:forward> action tags. For example, the following code passes the control to the JSP page process.jsp temporarily, with two additional parameters, user and sessionid.

Example:

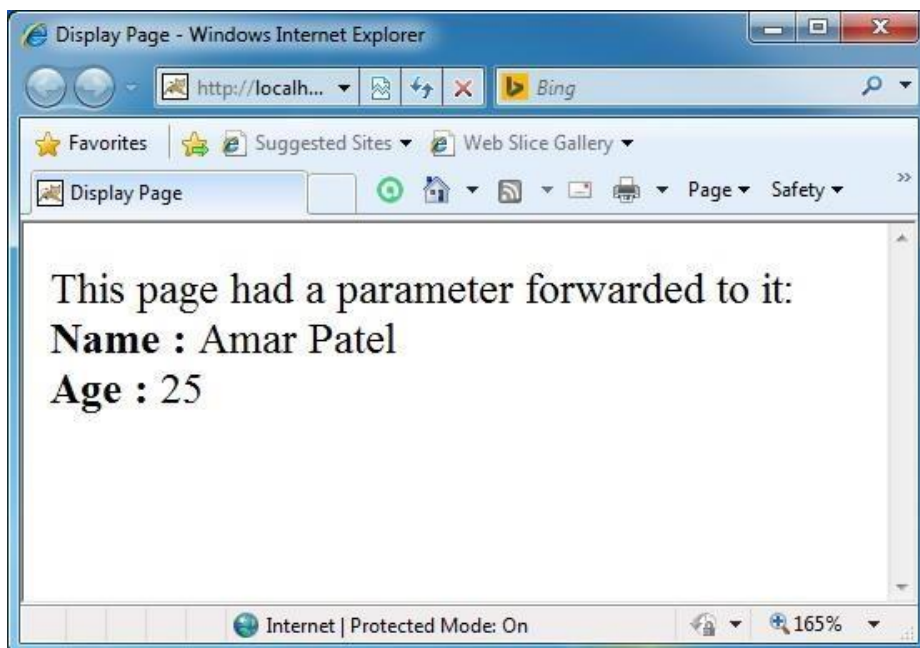
Forward.jsp:

```
<%@ page language="java" %>
<html>
  <head>
    <title> Forward page </title>
  </head>
  <body>
    <jsp:forward page="Parameters.jsp">
      <jsp:param name="uname" value="Amar Patel"/>
      <jsp:param name="age" value="25"/>
    </jsp:forward>
  </body>
</html>
```

Parameters.jsp:

```
<%@ page language="java" %>
<html>
  <head>
    <title> Display Page </title>
  </head>
  <body>
    This page had a parameter forwarded to it: <br>
    <b> Name : </b> <%= request.getParameter ("uname") %> <br>
    <b> Age : </b> <%= request.getParameter ("age") %>
  </body>
</html>
```

Output:



4.plugin

- The `<jsp:plugin>` action is used to generate an HTML file that can download the Java plug-in on demand and execute applets or JavaBeans.
- The `<jsp:plugin>` action generates the **embed** or **object** tag for the applet to be executed, depending upon the browser used.

Example:

Plugin.jsp:

```
<%@ page language="java" %>
<html>
    <head>
        <title> Plug in Page </title>
    </head>
    <body>
        <jsp:plugin type = "applet" code = "Message">
            <jsp:params>
                <jsp:param name="message" value="Hello World"
            /> </jsp:params>
            <jsp:fallback>
                <h1> Unable to Start Plug-in </h1>
            </jsp:fallback>
        </jsp:plugin>
    </body>
</html>
```

- The type attribute of the <jsp:plugin> tag specifies the type (bean or applet) of plug-in to be used.
- The code attribute specifies the name of the file (without extension) containing the byte code.
- The <jsp:params> and <jsp:param> actions are used to pass parameters to the plug-in.
- The <jsp:fallback> tag specifies the message to be displayed if the Java plug-in fails to start due to some unavoidable reason.

Message.java:

```
import java.applet.Applet;
import java.awt.Graphics;
public class Message extends Applet {
    String msg;
    public void init()
    {
        msg = getParameter ("message");
    }
    public void paint (Graphics g)
    {
        g.drawString (msg,20,20);
    }
}
```

This applet takes a single parameter message and displays its value at location (20,20) on the browser's screen, relative to the applet window.

5.UseBean

- The useBean action tag is used to locate or instantiate a JavaBean object into the JSP page.

The syntax is

<jsp:useBean id = "object_name" class = "class_name" scope = "page|request |session|application" />

- ✓ The id attribute refers the name of the object to be created.
- ✓ The attribute class specifies the name of the JavaBean class.
- ✓ The attribute scope specifies the area of visibility of the loaded bean.

Example:

circlearea.java:

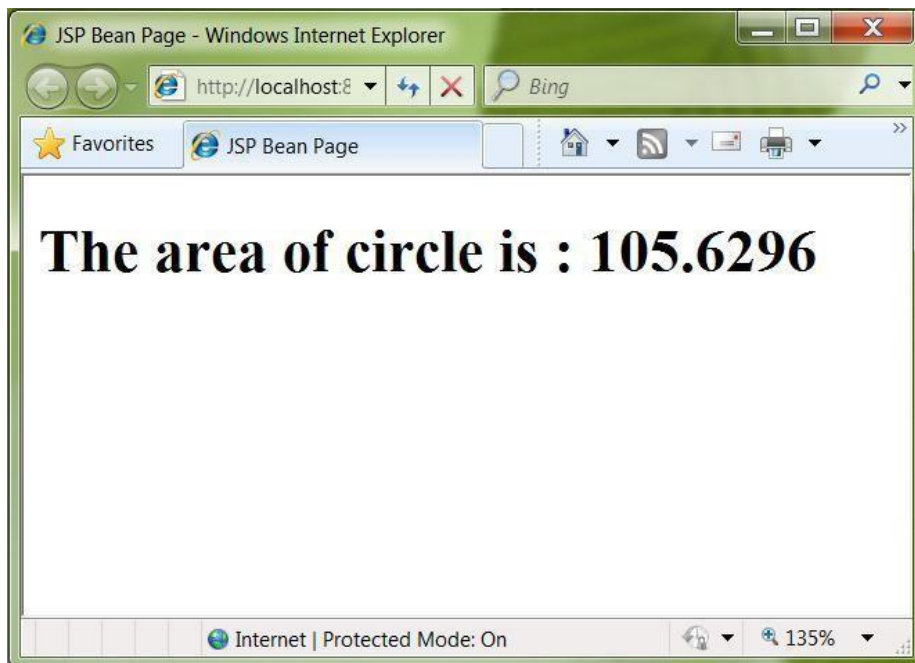
```
package usebeanpack;
public class circlearea
{
    static double pi = 3.14;
    public double area (double rad)
    {
```

```
double d = pi * rad * rad;  
return d;  
}  
}
```

usebean.java:

```
<%@ page language="java" %>  
<html>  
    <head>  
        <title> JSP Bean Page </title>  
    </head>  
    <body>  
        <jsp:useBean id="a" class="usebeanpack.circlearea" />  
        <%  
            double e = a.area(5.8);  
            out.print("<h1> The area of circle is : " + e + "</h1>");  
        %>  
    </body>  
</html>
```

Output:



15. Write a short notes on Tag Extensions in JSP?

(5 MARKS)

- JSP tag extension allow us to define and use custom tags in a JSP page exactly like other HTML/XML tags, using Java code.
- The JSP engine interprets them and invokes their functionality.
- JSP tag extension allows us to express complex functionalities in a simple and convenient way.

Tag Types

JSP specification defines the following tags, which we can create and use in JSP file.

1. Simple Tags

Tags with no body or no attributes.

Example:

```
<ukr:copyright>
```

2. Tags with attributes

Tags that have attributes but no body.

Example:

```
<ukr:hello name="ram">
```

3. Tags with body

Tags that can contain other tags, scripting elements and text between the start and end tags.

Example:

```
<ukr:hello>  
    <%= name %>  
</ukr:hello>
```

4. Tags defining Scripting Variables

Tags that can define scripting variables, which can be used in the scripts within the page.

Example:

```
<ukr:animation id="logo" />  
<%= logo.start(); %>
```

5. Cooperating Tags

Tags that cooperate with each other by means of named shared objects.

Example:

```
<ukr:tag1 id="obj1" />  
<ukr:tag2 name=" obj1" />
```

In this example, tag1 creates a named object called obj1, which is later used by tag2.

16. Explain in detail about Session Tracking with an example?

(11 MARKS)

- HTTP is a stateless protocol, the web server cannot remember previous requests. The web server cannot relate the current request with the previous one. This creates a problem for some applications that require a sequence of related request-response cycles.
- Examples include online examination systems, e-mail checking systems and banking applications.

A simple application to demonstrate session tracking using different methods.

- ✓ In this application, the server initially dynamically generates an HTML file to display the integer 0, with two buttons "prev" and "next".
- ✓ If the user clicks on the next button, the server sends the integer next to the one displayed.
- ✓ Similarly, when prev button is clicked, the server sends the integer previous to the one displayed.

The four ways to track the session are

1. Hidden fields
2. URL Rewriting
3. Cookies
4. Session API

1. Hidden Fields

An HTML hidden field is created using the <input> tag, with the type attribute hidden. For example, the following creates a hidden field.

```
<input type="hidden" name="user" value="chennai">
```

- The hidden fields is that web browsers do not display them, but send the name-value pair of each hidden field exactly like other input elements when the enclosing form is submitted.
- Hidden fields used to send information back and forth between server and client to track session, without affecting the display.

Hidden.jsp:

```
<%@ page language = "java" import = "java.util.*"
%> <html>
<head>
<title> Hidden Field Demo </title>
</head>
<body>
<%
    int current = 0;
    String last = request.getParameter ("int");
    String button = request.getParameter ("button");
```

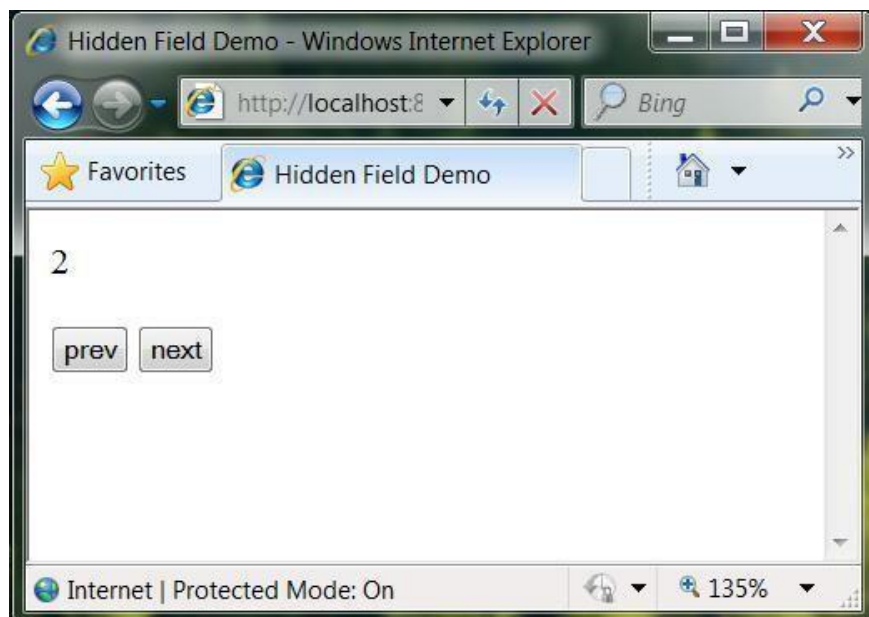
```

if (button != null)
{
    if (button.equals ("next"))
        current = Integer.parseInt (last) + 1;
    else
        current = Integer.parseInt (last) - 1;
}

out.println (current);
%>
<br>
<form name="myform" method="post">
    <input type="hidden" name="int" value="<%=current%>">
    <input type="submit" name="button" value="prev"> <input
    type="submit" name="button" value="next">
</form>
</body>
</html>

```

Output:



The HTTP POST method is used in the form and no action attribute is specified. When the user clicks any of the two submit buttons, the value of this hidden field is sent to the same JSP page. The JSP page extracts this value and understand this value was sent in response to the previous request. This way, the hidden field helps the JSP page to rememeber information sent earlier. It also determines the button whose click event generates this request. The JSP page can then easily calculate the value to be sent next, depending on the button clicked.

The advantage of this method is that it it does not require any special support from the client's side. Hidden fields are supported by all the browsers and underlying information propagation is absolutely transparent to the user.

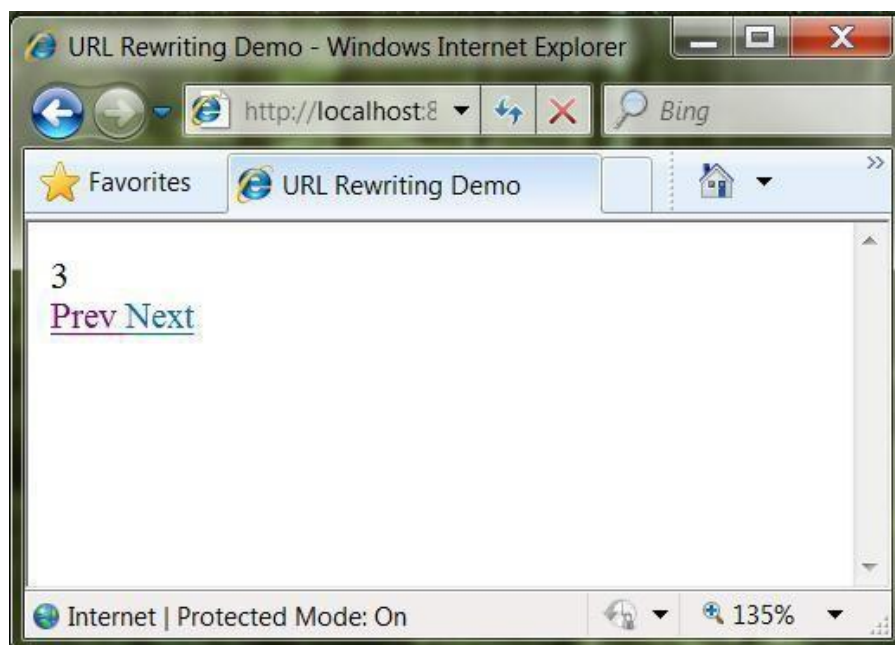
2. URL Rewriting

- URL Rewriting method to track sessions and is widely used.
- It does not require any special support from the browser.
- HTTP allows us to pass information using HTTP URL.
- The session information is appended to the URL rewriting.

UrlRewriting.jsp

```
<%@ page language="java" import="java.util.*" %>
<html>
    <head>
        <title> URL Rewriting Demo </title>
    </head>
    <body>
        <%
            int last = 0;
            String param = request.getParameter ("int");
            if (param != null)
                last = Integer.parseInt (param);
            out.println (last);
        %>
        <br>
        <a href="UrlRewriting.jsp ? int= <%=last-1%> "> Prev </a>
        <a href="UrlRewriting.jsp ? int= <%=last+1%> "> Next </a>
    </body>
</html>
```

Output:



When a user clicks on the “prev” hyperlink, the integer that should be sent by the JSP page for this request is appended to the URL. The JSP page simply extracts this information and sends it back with the new URLs for those two hyperlinks. A similar sequence of events happens when a user clicks on the “next” button.

3. Cookies

- Cookies method requires support from the web browsers. In this method, session management is represented by a named token called **cookie**.
- The JSP page sends this cookie to the web browser. The browser must be configured properly to accept it. Upon receiving a cookie, the web browser installs it.
- This cookie is then sent back to the server with the subsequent requests. The JSP page can then extract session information from this cookie. This way, the server can identify a session.
- By default, most browsers support cookies. However, due to security reasons, very often cookie support is disabled and in that case session management will not work properly.

Cookie.jsp:

```
<%@ page language="java" import="java.util.*" %>
<html>
  <head>
    <title> Cookie Demo </title>
  </head>
  <body>
    <%
      int current = 0;
      Cookie cookie = null;
      Cookie[] cookies = request.getCookies();
      if (cookies != null)
        for (int i=0; i<cookies.length; i++)
          if (cookies[i].getName().equals("last"))
            cookie = cookies[i];
      if(cookie != null)
      {
          String button = request.getParameter("button");
          if(button!=null)
          {
              if(button.equals("next"))
                  current = Integer.parseInt(cookie.getValue())+1;
              else
                  current = Integer.parseInt(cookie.getValue())-1;
          }
      }
```

```

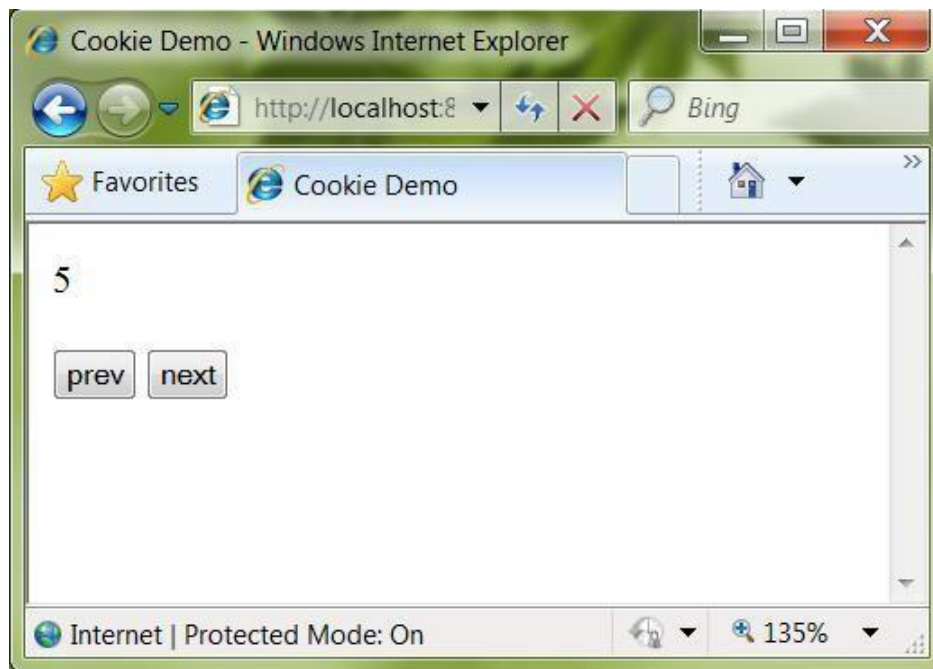
    }

    Cookie ck = new Cookie("last", String.valueOf(current));
    response.addCookie(ck);
    out.println(current);

%>
<br>
<form name="myform" method="post">
    <input type="submit" name="button" value="prev">
    <input type="submit" name="button" value="next">
</form>
</body>
</html>

```

Output:



The JSP page first looks for a cookie named "last". If it finds the cookie, the value of the cookie is obtained using the `getValue()` method. Depending on the button clicked, the integer to be sent next is calculated and stored in the variable `current`. This integer, together with a new cookie with the name "last" having the current value set is sent to the web browser. However, if the JSP page does not find any cookie with the name "last", which happens during the first time, it sends a new cookie having the value 0, which is the initial value of the variable `current`.

4.Session API

- JSP technology provides a higher level approach for session tracking. The basic building block of this Session API is the `javax.servlet.http.HttpSession` interface.
- The `HttpSession` objects are automatically associated with the client by the cookie mechanism.
- An `HttpSession` object that persists during a session, until it times out or it shutdown by the JSP page participating in this session.

In a JSP page, an HTTP session is created using session attribute of the page directive

```
<%@ page session= " true | false " %>
```

In the `HttpSession` object, we can store and retrieve key-value pair using `setAttribute()`, `getAttribute()` and `getAttribute()` methods.

1. Object `getAttribute (String key)`
2. Enumeration `getAttributeName();`
3. void `setAttribute (String key, Object value);`

The JSP engine uses the cookie mechanism to keep track of sessions. A session object is associated with usually a long alphanumeric ID. This session ID is sent to the client as a cookie with the name `JSESSIONID`, when the client makes a request for the first time using the HTTP response header `Set-Cookie`, as follows

Set-Cookie: JSESSIONID=g52d15acu325dlw532h234

For subsequent requests, the client sends this cookie back to the server using the HTTP request header `Cookie` as follows:

Set-Cookie: JSESSIONID=g52d15acu325dlw532h234

The server can then identify that this request has come from the same client. However, if the client does not accept cookies, this mechanism fails. In case, URL rewriting mechanism, where each URL must have the session ID appended. The `HttpServletResponse` object provides methods to append this session ID automatically. These method identify whether the client is configured to accept cookies. They append the session ID only if the client does no accept cookies.

```
java.lang.String      encodeURL(java.lang.String);  
java.lang.String encodeRedirectURL(java.lang.String);
```

The `encodeRedirectURL()` method is used for the `sendRedirect()` and `encodeURL()` method is used for the rest to create such URLs.

SessionAPL.jsp

```
<%@ page language="java" import="java.util.*" %>  
<html>  
  <head>  
    <title> Session API Demo </title>  
  </head>
```

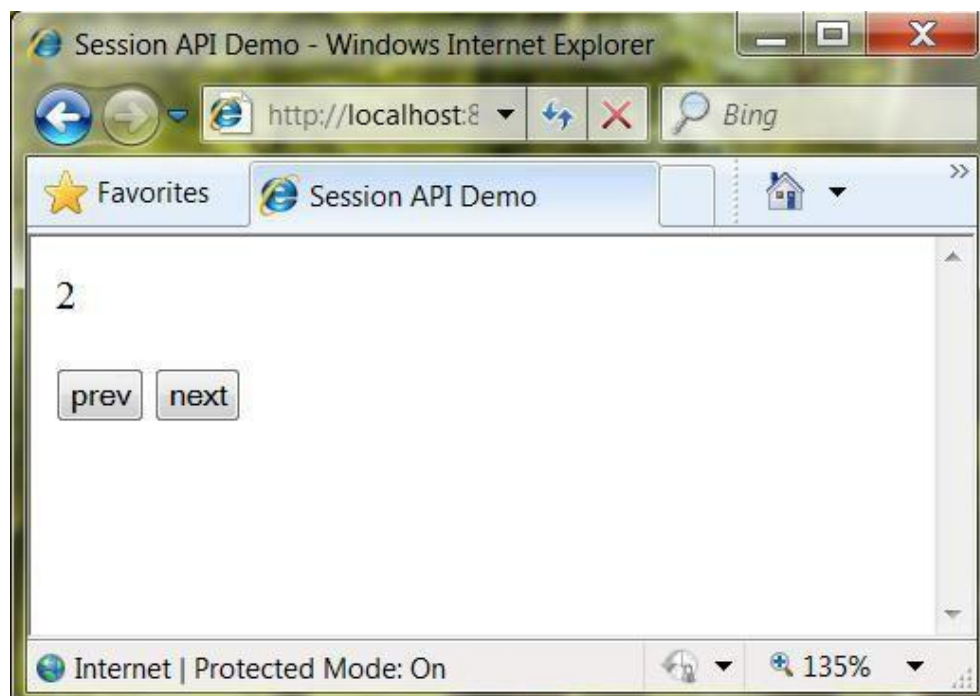
```

<body>
<%
    int current = 0;
    String last = (String) session.getAttribute("last");
    if (last != null) {
        String button = request.getParameter("button");
        if (button != null)
        {
            if (button.equals("next"))
                current = Integer.parseInt(last)+1;
            else
                current = Integer.parseInt(last)-1;
        }
    }

    session.setAttribute ("last",
        String.valueOf(current)); out.println(current);
    %>
<br>
    <form name="myform" method="post">
        <input type="submit" name="button" value="prev">
        <input type="submit" name="button" value="next">
    </form>
</body>
</html>

```

Output



17. Explain how are the JSP requests handled with an example?**(6 MARKS) (APR 2016)**

- JSP page wants to pass the control to another server-side program for further processing.
- To send information to web server, we can use two methods: GET Method and POST Method.
- The GET method sends the encoded user information separated by the ? character appended to the page URL.
- The GET method is the default method to send information to web server. Since the GET method appends plain text string to the URL. We should avoid using GET method to send password or other sensitive information to the server.
- The GET method also has size limitation. We can send only 1024 characters in a request string. This information sent is accessible `getQueryString()` and `getParameter()` methods of request object.
- POST method is more reliable method of sending information to the server. This method sends the information as a separate message.
- JSP handles this type of requests using `getParameter()` method to read simple parameters and `getInputStream()` method to read binary data stream coming from the client.

JSP handles form data using the following methods.

Method	Description
<code>getParameter()</code>	returns the value of a form parameter.
<code>getParameterValues()</code>	returns multiple values, for example checkbox.
<code>getParameterNames()</code>	returns a complete list of all parameters.
<code>getInputStream()</code>	read binary data stream coming from the client.

GET parameters from URL

- The following code shows how to get parameters using GET method.
- The following code uses the `getParameter()` method to read parameters.

Main.jsp:

```
<html>
  <body>
    <b>First Name:</b><%= request.getParameter("first_name")%><br/>
    <b>Last Name:</b><%= request.getParameter("last_name")%>
  </body>
</html>
```

- Save the code above as `main.jsp`. And copy the file to the Tomcat deployment folder.
- First we would type in the parameters and their value directly in the URL. Enter the following URL in your browser address bar.

`http://localhost:8080/main.jsp?first_name=James&last_name=Smith`

We can use the following html form to pass the values to JSP main.jsp page.

```
<html>
  <body>
    <form action="main.jsp" method="GET">
      First Name: <input type="text" name="first_name"> <br />
      Last Name: <input type="text" name="last_name"
    /> <input type="submit" value="Submit" />
    </form>
  </body>
</html>
```

POST Method Example with Form

Create an html page as follows and use POST methods for the form. Save it as Hello.html. <html>

```
<body>
  <form action="main.jsp" method="POST">
    First Name: <input type="text" name="first_name"> <br />
    Last Name: <input type="text" name="last_name"
  /> <input type="submit" value="Submit" />
  </form>
</body>
</html>
```

Below is main.jsp JSP program to handle input from by web browser using GET or POST methods. <html>

```
<body>
  <b>First Name:</b><%= request.getParameter("first_name")%>
  <br/> <b>Last Name:</b><%= request.getParameter("last_name")%>
</body>
</html>
```

Passing Checkbox Data to JSP

- The following code shows how to use the JSP page to get the checkbox data from a form.
- Here is the html form code. It uses the POST method and calls the main.jsp

page. <html>

```
<body>
  <form action="main.jsp" method="POST" target="_blank">
    <input type="checkbox" name="a" checked="checked" /> A
    <input type="checkbox" name="b" /> B
```

```

        <input type="checkbox" name="c" checked="checked" />
        C <input type="submit" value="Select Subject" />
    </form>
</body>
</html>

```

The following is the main.jsp code to handle input for checkbox button.

```

<html>
    <body>
        <b>A Flag:</b><%= request.getParameter("a")%>
        <b>B Flag:</b><%= request.getParameter("b")%>
        <b>C Flag:</b><%= request.getParameter("c")%>
    </body>
</html>

```

Reading All Form Parameters

- The following code shows how to use `getParameterNames()` method from `HttpServletRequest` to read all the available form parameters.
- This method returns an Enumeration object that contains the parameter names.

```

<%@ page import="java.io.*,java.util.*" %>
<html>
<body>
<%
    Enumeration paramNames = request.getParameterNames();
    while(paramNames.hasMoreElements()) {
        String paramName = (String)paramNames.nextElement();
        out.print("<b>" + paramName + " :</b>\n");
        String paramValue = request.getHeader(paramName);
        out.println(paramValue);
    }
%>
</body>
</html>

```


Example:

Index.html

```
<html>
  <head>
    <title> ARITHMETIC OPEARTIONS </title>
  </head>
  <body>
    <h1 align="center"> <font color="green"> MATHEMATICAL FUNCTIONS </font>
  </h1> <form name="calc" method="post" action="calculate.jsp">
    <table align="center" cellpadding="5%" cellspacing="5%" border="3"
      bordercolor="#FF0000"> <tr>
      <td> ENTER THE NO1 <input type="text" name="t1"> </td>
    </tr>
    <tr>
      <td> ENTER THE NO2 <input type="text" name="t2"> </td>
    </tr>
    <tr>
      <td> <input name="radio" type="radio" value="add"> ADDITION </td>
    </tr>
    <tr>
      <td> <input name="radio" type="radio" value="sub"> SUBTRACTION </td>
    </tr>
    <tr>
      <td> <input name="radio" type="radio" value="mul"> MULTIPLICATION </td>
    </tr>
    <tr>
      <td><input name="radio" type="radio" value="div"> DIVISION </td>
    </tr>
  </table> <br> <br>

  <table align="center" cellpadding="5%" cellspacing="5%" border="3"
    bordercolor="#FF0000"> <tr>
      <td> <input type="submit" name="b1" value="Submit here"> </td>
      <td> <input type="reset" name="re1" value="Reset here"> </td>
    </tr>
  </table>
</form>
</body>
</html>
```

calculate.jsp:

```
<%@ page language="java"%>
<%@ page import="java.lang.*" %>
<html>
    <head>
        <title>Calculate Operation</title>
    </head>
    <body bgcolor="orange">
        <%
            /*Retrieving the value of radio button */
            String str=request.getParameter("radio");
            String str1=request.getParameter("t1");
            String str2=request.getParameter("t2");
            String output="";

            /* declaring integer variable for performing calculation */
            int num1=0;
            int num2=0;
            int num3=0;

            /*Converting String values into Integer */
            num1=Integer.parseInt(str1);
            num2=Integer.parseInt(str2);

            //Checking which mathematical function is selected by the user
            if(str.equals("add"))
            {
                num3=num1+num2;
                output="Addition";
            }
            if(str.equals("sub"))
            {
                num3=num1-num2;
                output="Subtraction";
            }
            if(str.equals("mul"))
            {
                num3=num1*num2;
                output="multiplication";
```

```

    }
    if(str.equals("div"))
    {
        num3=num1/num2;
        output="Division";
    }
}
%>

```

```

<%-- Displaying the option selected --%>

```

```

<h1 align=center> <font color="blue"> The Selected mathematical function by you is:

```

```

<%= output%> </font> </h1> <br>

```

```

<%

```

```

    // Displaying the result

```

```

    Integer in =new Integer(num3);

```

```

    out.println("<h3 align=center> <font color=green> The Result is : " +in.toString()+"

```

```

    </font> </h3>");

```

```

%>

```

```

</body>

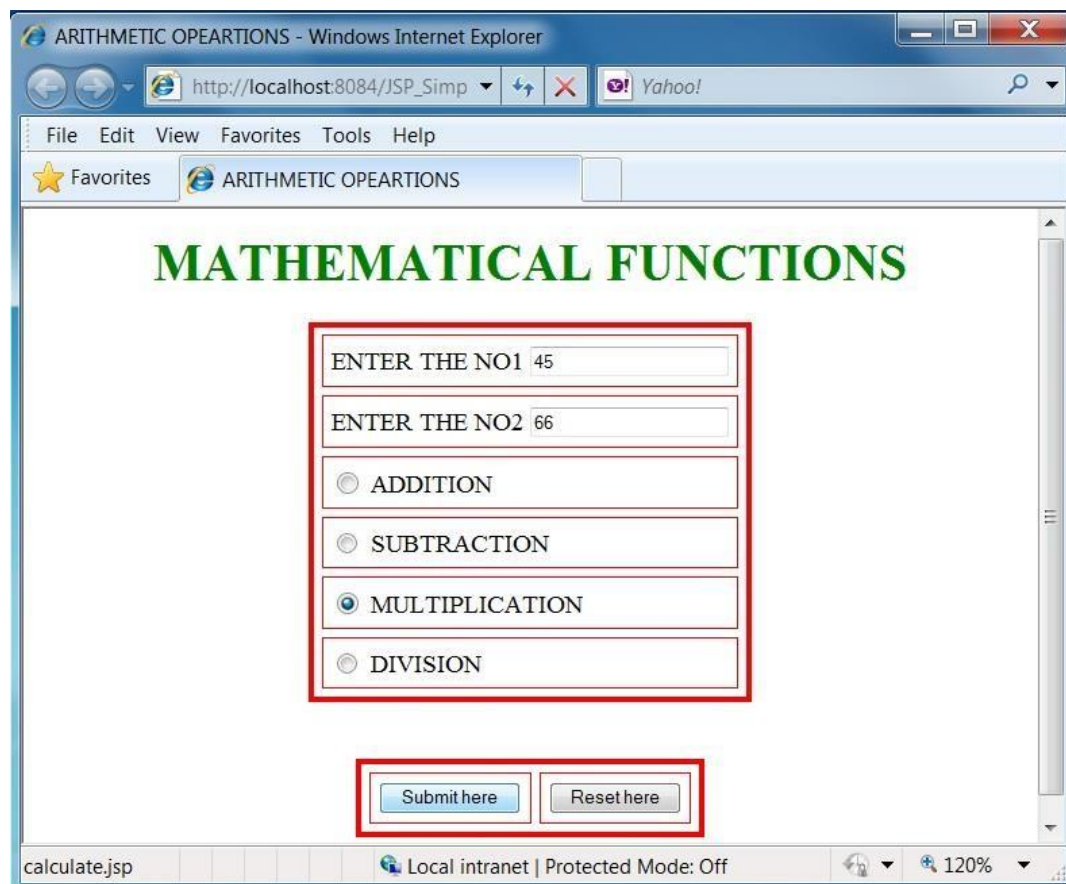
```

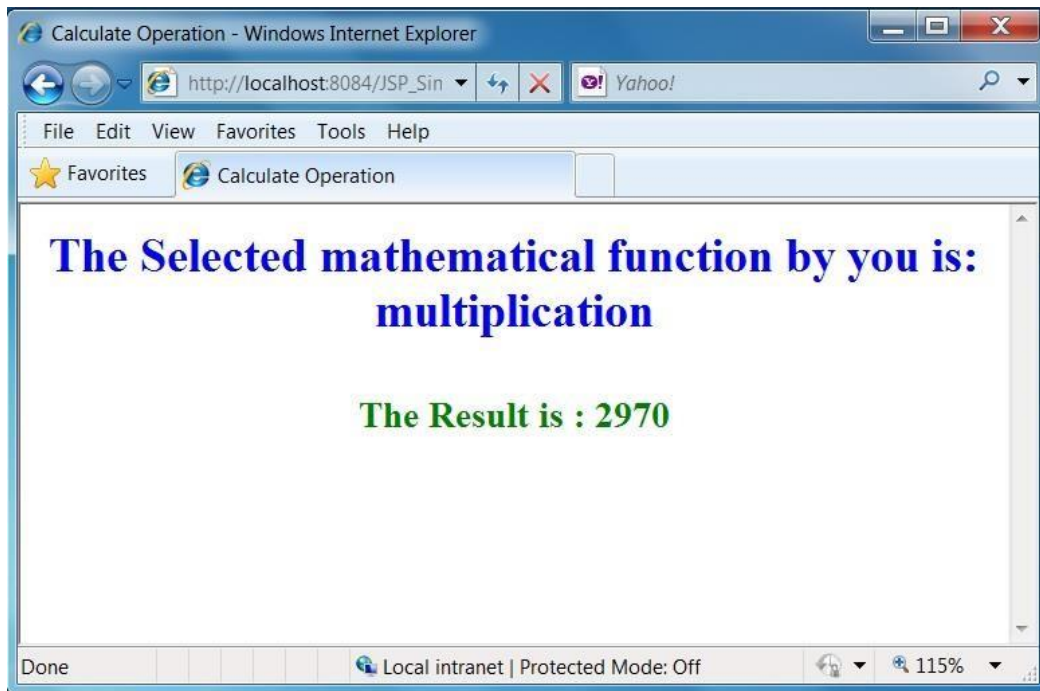
```

</html>

```

Output:





18. Explain the Database Connectivity in JSP?

(11 MARKS) (APR 2017)

Java Database Connectivity (JDBC) allows us to access databases through Java programs. It provides Java classes and interfaces to SQL and PL/SQL statements, process results and performs other operations common to database.

Since Java Server Pages can contain Java code embedded in them, it is also possible to access databases from Java Server Pages. The classes and interfaces for the database connectivity are provided as a separate package, java.sql.

JDBC Drivers

A Java application can access almost all types of databases such as relational, object, and object relational. The access to a specific database is accomplished using a set of Java interfaces, each of which is implemented by different vendors differently. A Java class that provides interfaces to a specific database is called **JDBC driver**.

Each database has its own set of JDBC drivers. JDBC drivers provided by database vendors convert database access requests to database-specific APIs.

Types of JDBC Drivers

JDBC drivers are classified into four categories

1. JDBC-ODBC bridge
2. Native-API, Partly Java
3. Middleware, Pure Java
4. Pure Java Driver

1. JDBC-ODBC bridge (Type 1)

JDBC-ODBC drivers cannot talk to the database directly. It needs an intermediate ODBC (Open DataBase Connectivity) driver, with which it forms a kind of bridge. The drivers translates JDBC function calls to ODBC method calls. ODBC make use of native libraries of the operating system and is hence platform-dependent.

2. Native-API (Type 2)

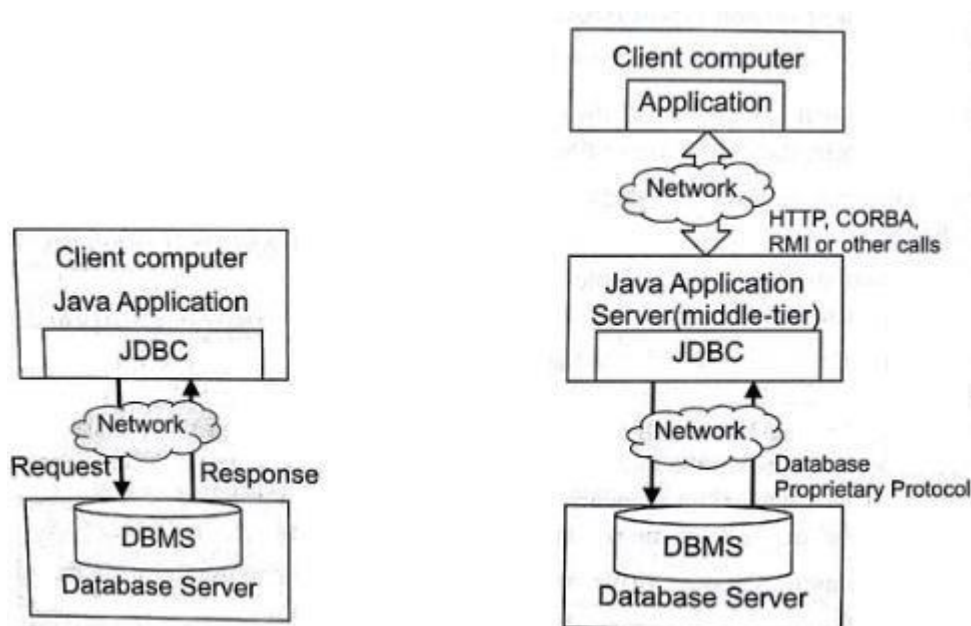
This driver does not forward the JDBC call to the ODBC driver. Instead, it translates JDBC calls to database-specific native API calls. This driver is not a pure Java driver, as it interfaces with non-Java APIs that communicates with the database. This is faster than ODBC driver, as it interfaces directly with the database through the native APIs.

3. Middleware, Pure Java (Type 3)

The JDBC driver forwards the JDBC calls to some middleware server using a database-independent network protocol. The middleware server acts as a gateway for multiple database servers and can use different database-specific protocols to connect to different database servers. This intermediate server sends each client request to a specific database. The results are then sent back to the intermediate server, which in turn sends the result back to the client. This approach hides the details of the connection to the database servers and makes it possible to change the database servers without affecting the client.

4. Pure Java Driver (Type 4)

This types of drivers communicate with the database directly by making socket connections. Since it talks with the database server directly, no other subsidiary driver is needed.



JDBC architecture (i) two tier (ii) three-tier

JDBC classes and Interfaces

DriverManager	Connnection	Statement
PreparedStatement	ResultSet	ResultSetMetaData
CallableStatement	DatabaseMetaData	

Basic Steps:

The following basic steps to work with JDBC

1. Loading a Driver
2. Making a connection
3. Create the statement object
4. Executing an SQL statement
5. Close the connection object

1. Loading a Driver:

➤ **Register the driver class**

The **forName()** method of Class is used to register the driver class. This method is used to dynamically load the driver class.

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

The method **registerDriver()** in turn registers the driver with the driver manager.

2. Making a connection

- Once a driver is instantiated and registered with the driver manager, the connection to the database established using methods provided by the DriverManager class.
- The **getConnection()** method of DriverManager class is used to establish connection with the database.

The following method are available on the DriverManager class to establish a connection.

```
public static Connection getConnection (String url, String login, String  
passwd) public static Connection getConnection (String url)  
public static Connection getConnection (String url, properties)
```

A database connection can be established using this URL as follows:

```
Connection con = DriverManager.getConnection  
("jdbc.oracle.thin:@localhost:1521:xe", "system", "password");
```

3. Create the statement object

The **createStatement()** method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.

```
Statement stmt = con.createStatement();
```

4. Executing an SQL statement

The **executeQuery()** method of Statement interface is used to execute queries to the database. This method returns the object of **ResultSet** that can be used to get all the records of a table.

```
ResultSet rs = stmt.executeQuery("select * from emp");
```

The **executeUpdate()** method is used to execute DDL (Create, Alter, and Drop), DML (Insert, Delete, Update etc..) and DCL statements. An SQL command changes the database, the **executeUpdate()** method is used. The return value of this method is the number of rows affected.

5. Closing the connection object

The **close()** method of Connection interface is used to close the connection.

```
con.close();
```

19. Write a short notes on J2EE?

(5 MARKS)

Today, in the fast-moving and highly demanding world e-commerce and Information Security (IT), there is a tremendous need for low-cost, distributed applications for enterprise. These distributed enterprise applications must be designed, implemented, and deployed in less time, with few resources and greater speed.

The **Java 2 Enterprise Edition (J2EE)** provides a component-based technology for the design, development, assembly and deployment of low-cost and fast-track enterprise applications. The purpose of J2EE is to simplify the design and implementation of distributed enterprise applications.

J2EE offers the following fundamentals are

- Multi-tiered distributed application model
- Reusable components
- Flexible transaction control
- Web services support through integrated data interchange on XML-based open standards and protocols
- Unified security model

Since J2EE solutions are basically Java-based, they are platform-independent and are not tied to a specific vendor or customer. Customers and vendors are free to select from a wide variety of products and components.

J2EE is not a single technology; it consists of a large set of technologies.

- ✓ Enterprise Java Beans (EJB)
- ✓ Java Servlets
- ✓ Java Server Pages (JSP)
- ✓ Java Message Service (JMS)

- ✓ Java Naming and Directory Interface (JNDI)
- ✓ Java-XML
- ✓ Java Mail
- ✓ Java DataBase Connectivity (JDBC)
- ✓ Remote Method Invocation (RMI)
- ✓ CORBA
- ✓ RMI-IIOP

20. Write about the steps involved in creating simple Beans in detail. (11 MARKS) (NOV 2016)

- JavaBean technology is a Java-based technology to design and develop reusable and platform-independent software components. These components can then be integrated virtually in all Java-compatible applications. JavaBean components are known as **beans**.
- The appearance and features of a bean can be changed or customized using builder tools such as BeanBox and NetBeans.

A bean class is nothing but a usual Java class with the following requirements:

- It must have a public no-argument constructor.
- It has public accessor methods to read and write properties.
- It should implement Serializable or Externalizable interface.

Example

State.java

```
import java.io.Serializable;

public class State implements Serializable
{
    protected String state="off";
    public String getState()
    {
        return state;
    }

    public void setState(String state)
    {
        this.state=state;
    }
}
```

JavaBeans have a GUI representation. For example, button, calculator, and calendar beans are expected to have a visual representation. Invisible beans include a spell checker, random number generator and temperature monitor.

Advantages of JavaBeans

JavaBeans mission statements:

“Write once, run anywhere, use everywhere”.

Steps involved in creating simple Beans

1. Creating a simple Bean
2. Compiling and saving the Bean into a Java Archive (JAR) file
3. Loading the Bean into the ToolBox
4. Dropping a Bean instance into the BeanBox
5. Inspecting the Bean's properties, methods and events
6. Generating an introspection report

21. Write a short notes on EJB?

(5 MARKS)

- Enterprise JavaBeans (EJB) is a Java-based component architecture for design and development of world-class distributed modular enterprise applications.
- EJB are platform independent, but it can run in any application server that implements EJB specifications.

The EJB component architecture integrates several enterprise-level requirements such as

- distributions,
- transactions,
- security,
- messaging,
- persistence and
- Enterprise Resource Planning (ERP) systems.

There are three kinds of beans in the EJB architecture:

1. Session Beans
2. Entity Beans
3. Message Driven Beans

22. List out the applications object in cookies.**(5 MARKS)****(NOV 2017)**

The application implicit object encapsulates a view of the collection of all servlets, JSP pages, HTML pages, and other resources in a Web application. This object implements the `javax.servlet.ServletContext` interface and is automatically constructed at the beginning of the `_jspService()` method. It provides information about the server version, any application-wide initialization parameters, and the absolute paths of resources within the application. This object also provides a means for logging messages.

Method	Description
Enumeration <code>getAttributeNames()</code>	Returns an enumeration of the names of all objects stored in the servlet context.
Object <code>getAttribute(String name)</code>	Returns an object with the specified name that was stored with the application's <code>setAttribute()</code> method.
String <code>getInitParameter(String name)</code>	Returns the value of the specified application-wide initialization parameter.
Enumeration <code>getInitParameterNames()</code>	Returns an enumeration of the names of all application-wide initialization parameters.
String <code>getRealPath(String path)</code>	Converts a path in the context of the Web application to an absolute path in the file system, if possible.
URL <code>getResource(String path)</code>	Returns the URL (if any) mapped to the specified path in the application. The path must begin with a "/" and is relative to the root of the application.
InputStream <code>getResourceAsStream(String path)</code>	Similar in operation to <code>getResource()</code> , but returns an opened input stream to the resulting URL.
void <code>log(String msg)</code>	Writes a message to the log file associated with this application.